

TED UNIVERSITY
Department of Computer Engineering

CMPE 492 – Senior Design Project II

TEDU Guide

AI-Powered Intelligent Campus Navigation System

Low-Level Design Report

Deliverable D1 (first version) | Fall 2026–2027 | Version 1.0

Project website: https://tedu-guide.vercel.app/

Team Members
Taci Furkan Gökalp · Elif Nazlı Özer · Fatima Nasir Muhammad

Advisor & Jury Members
Advisor: Tolga Kurtuluş Çapın
Jury Member 1: Mehmet Evren Coşkun
Jury Member 2: Müslüm Bozyiğit

Submission date: October 16, 2026

Document Control

Item	Value
Document	Low-Level Design Report (LLD), CMPE 492 – first version (deliverable D1, due 16 October 2026). The second version (deliverable D3, due 23 October 2026) will incorporate the supervisor's feedback.
Refines	High-Level Design Report D6 (15 May 2026), Analysis Report D4 (10 April 2026), Specification Report D2 (27 March 2026) of CMPE 491.
Status	Draft for advisor review. Latency, accuracy, size and capacity figures in this document are **design targets or estimates**, not measurements; measured values will appear in the Test Plan and the Final Report.
Prepared by	All team members. Section owners are listed in Section 2.3 (Package ownership and dependencies).
Reading guide	Sections 1–2: why the design looks as it does and how it is organised. Section 3: one subsection per component, with the exact interfaces. Sections 3.11–3.16: cross-cutting design, operations, traceability, testability, impact, risks. A reader who only wants the key decisions can read Sections 1.1–1.3 and 1.7.

Revision history

Version	Date	Description
0.9	2026-10-09	First complete draft (packages, class interfaces, data model, algorithms, standards, traceability).
1.0	2026-10-10	Revised for submission on 16 October: positioning moved to the device (privacy by architecture), realistic WiFi scan cadence with dead reckoning and map matching, LLM time budget aligned with NFR-01, domain classes and answer composition, radio-map and navigation-graph endpoints, roles, analysis-to-design mapping, design-principle validation, testing hooks, impact and risk tables, corrected figures and tables.
2.0 (planned)	2026-10-23	Second version after supervisor feedback (deliverable D3).

Table of Contents

Document Control 2
1. Introduction 4
1.1 Purpose and scope 4
From analysis objects to design classes 4
What changed with respect to D6 4
1.2 Object design trade-offs 5
1.3 Interface documentation guidelines 6
1.4 Engineering standards 7
1.5 Design principles and how they are validated 8
1.6 Design patterns 8
1.7 Creativity of the design 9
1.8 New tools and technologies 9
1.9 Definitions, acronyms and abbreviations 10
2. Packages 11
2.1 Layer and package overview 11
2.2 Repository layout 11
2.3 Package ownership and dependencies 11
2.4 Deployment view 12

2.5 Configuration 13
3. Class Interfaces 14
3.1 Data model (SS-07) 14
3.2 REST API contract (SS-02) 15
3.3 Natural-language subsystem (SS-03) 17
3.3.1 Text normalisation 17
3.3.2 Parser interfaces 17
3.3.3 Entity resolution, validation and answer composition 18
3.4 Positioning subsystem (SS-04) 19
3.4.1 Radio map format and delivery 19
3.4.2 Provider interface and manager (device, TypeScript) 19
3.4.3 k-NN estimator (device) 20
3.4.4 Fusion between scans: motion, filter and map matching 20
3.4.5 Training, evaluation and publishing (server) 21
3.5 Routing subsystem (SS-05) 22
3.5.1 Graph model 22
3.5.2 Algorithm 22
3.6 Campus data and search (SS-07, FR-07) 23
3.6.1 Domain classes 23
3.6.2 Repositories and search 23
3.7 Administration and authentication (SS-06) 24
3.8 API layer and orchestration 24
3.9 Client application (SS-01) 26
3.9.1 Session state machine 26
3.9.2 Modules 27
3.9.3 Consent and permissions 28
3.9.4 Map viewer 28
3.9.5 Offline behaviour (FR-10) 28
3.9.6 Screen inventory and accessibility criteria 28
3.10 Map data pipeline (offline tools) 29
3.11 Cross-cutting design 29
3.11.1 Security (NFR-05) 29
3.11.2 Privacy and data flow (NFR-05, NFR-06) 30
3.11.3 Performance and capacity (NFR-01, NFR-02, NFR-03) 30
3.11.4 Error handling matrix 31
3.12 Build, deployment and operations 31
3.13 Requirements traceability 32
3.14 Design for testability 32
3.15 Impact and contemporary issues (preview of the Final Report) 33
3.16 Implementation responsibilities and risks 34
4. Glossary 35
5. References 36

1. Introduction

1.1 Purpose and scope

This report refines the high-level design (D6) of TEDU Guide into an implementable low-level design. D6 decomposed the system into seven subsystems (SS-01 … SS-07) and specified their responsibilities. Through object design this document closes the gap between the application objects found during analysis (D4) and the off-the-shelf components we build on (React, Leaflet, FastAPI, SQLAlchemy, PostgreSQL/PostGIS, scikit-learn, NetworkX, the OpenAI API, Capacitor). It identifies additional solution objects, adjusts the off-the-shelf components, and specifies every subsystem interface and class precisely enough that each team member can implement a component without further design meetings.

The scope is the complete software system: the browser/PWA client (also packaged as an Android application), the REST backend, the positioning subsystem (on the device, plus calibration tools on the server), the natural-language subsystem, the routing subsystem, the campus database and the administration interface, plus the offline tool chain that turns TED University's official wayfinding maps into floor-plan and navigation-graph data.

From analysis objects to design classes

The class model of D4 contains ten application objects. The table shows where each one lives in the low-level design, and which solution objects were added around it.

D4 object	Design classes (Section)	Remarks
`User`, `Student`, `Visitor`, `Administrator`	No server-side user class for guests (FR-12). `AdminUser` with a role, `AuthService` (3.7). On the client the user is represented by settings and consent state (3.9).	Visitors and students are indistinguishable by design: no profile, no tracking.
`NavigationSession`	Client: `NavigationController` / `sessionReducer` finite-state machine (3.9.1). Server: none (stateless).	The session moved to the client so that the server never holds a position (T2).
`NLPEngine`	`HybridIntentParser` (with `FastPathParser`, `OpenAIIntentParser`, `RuleIntentParser`), `EntityResolver`, `ValidationLayer`, `ResponseComposer` (3.3).	The D4 class was too large; it was split by responsibility.
`WiFiFingerprint`	`FingerprintPoint`, `FingerprintSample`, `RadioMap`, `LocalKnnEstimator` (device), `RadioMapBuilder`, `AccuracyEvaluator` (server) (3.4).	Estimation runs on the device; building and testing the map runs on the server.
`Route`	`Route`, `RouteStep`, `RoutePlanner`, `NavGraph`, `CostModel`, `InstructionGenerator` (3.5).	Stairs and lifts are ordinary graph edges.
`CampusDatabase`	`CampusRepository`, `NavRepository`, `RadioMapRepository`, `AdminRepository`, `SearchService` (3.6).	No component writes SQL except repositories.
`Location`, `Room`, `Office`, `Facility`	`Location` entity with `type`/`subtype`, `Person`, `OpeningHours`, `Building`, `Floor` (3.6.1).	The three D4 subclasses are modelled by `type` plus type-specific data, which keeps the schema simple.

What changed with respect to D6

Writing the low-level design exposed several points in D6 that cannot be implemented as written. We list them here because the rest of the document depends on the corrections; each is justified in Section 1.2.

#	D6 statement	Problem found	Low-level decision
1	"RSSI collector uses the browser Network Information API" (SS-01, §3.1)	No mobile browser exposes BSSID/RSSI of surrounding access points; the Network Information API only reports the effective connection type. iOS does not permit WiFi scanning for ordinary apps at all.	`PositioningProvider` abstraction (Strategy). WiFi fingerprinting runs through a Capacitor plug-in on Android; QR anchors and manual pin-drop cover every other device.

#	D6 statement	Problem found	Low-level decision
2	4 Gunicorn/Uvicorn workers with an in-memory session dictionary (§3.6.3, §3.4)	Worker processes do not share memory; a location stored by one worker is invisible to the next request.	The server is stateless. It never receives a position that it must remember (T2).
3	Geometry columns in EPSG:4326 (WGS84)	Indoor plans are not geo-referenced; latitude/longitude would add meaningless distortion.	Local metric coordinates (SRID 0, metres) per floor; Leaflet `CRS.Simple` (T3).
4	FR-07 requires faculty names, but the schema has no faculty table	Person queries (“Prof. Çapın’s office”) cannot be answered.	New `people` table with folded search text; `locations.door_node_id` links rooms to the graph.
5	`rssi_fingerprints` stores mean and standard deviation only	Training, cross-validation and re-calibration need raw samples.	`fingerprint_points` + `fingerprint_samples` (raw vectors, JSONB).
6	Admin login with “university credentials”	No SSO/LDAP integration is planned or approved.	Local admin accounts (Argon2id) with roles; JWT access token + rotating opaque refresh token; SSO is future work.
7	The client sends the RSSI vector with every query; the server runs the k-NN (SS-02, SS-04)	Every user’s signal fingerprint, and therefore their location, would reach the server every few seconds; Android also limits scans to four per two minutes, so a 3-second polling loop is impossible on normal phones.	k-NN runs **on the device** from a downloaded, versioned radio map (T14). The server only builds, tests and publishes radio maps. Scans arrive about every 30 s; dead reckoning bridges the gap (T15).
8	GPT-4 timeout not specified; “validation layer” described only as a cross-check	A 4-second LLM wait cannot meet NFR-01 (2 s). Free-text LLM output could still leak into answers.	LLM limit 1.5 s with a circuit breaker (T16); answers are composed from templates and database values only (T5).

1.2 Object design trade-offs

The table records the principal design decisions, the alternatives we rejected and why. Decisions are ordered by their influence on the rest of the design.

ID	Decision	Alternatives considered	Rationale / consequence
T1	**Positioning is a pluggable provider**: `wifi` (Android via Capacitor), `qr`, `manual`, `simulated`.	(a) Browser-only WiFi fingerprinting; (b) BLE beacons (new hardware); (c) campus WLAN-controller positioning.	(a) is technically impossible (Section 1.1, row 1). (b) violates the economic constraint (no hardware). (c) needs IT cooperation we do not have; kept as a future provider. The abstraction lets the demo run on every device and lets us report honest accuracy per method.
T2	**Stateless backend**; no table, cache or log holds a user position.	In-memory session dictionary; Redis session store; database sessions.	Removes cross-worker inconsistency, removes the need for Redis (zero budget), makes horizontal scaling trivial (NFR-02) and makes privacy auditable.
T3	**Local metric coordinates** per floor, SRID 0, Leaflet `CRS.Simple`.	WGS84 with geo-referenced plans; raster tiles.	Distances in metres are directly usable by routing and by k-NN error metrics. Avoids geo-referencing effort and distortion.
T4	**Hybrid NLP chain**: fast path → cache → LLM → rule parser, then mandatory database validation.	LLM only; rules only; fine-tuned local model.	LLM-only misses the 2 s budget (NFR-01) for trivial queries and fails when the API is down. Rules-only cannot handle free-form Turkish/English. The chain gives p95 latency below 2 s, graceful degradation, and bounded API cost.
T5	**Zero-hallucination invariant**: every id, name and number in a response comes from the database; answer text comes from templates (`ResponseComposer`).	Trust the LLM output after prompt constraints; let the LLM write the answer.	Required by D2 §1.3 (algorithmic accountability). A structural property of `ValidationLayer` and `ResponseComposer`, so it can be tested exhaustively (property test). Cost: answers are less conversational.
T6	**k-NN** (distance-weighted) for fingerprinting.	Random forest, SVM, CNN/LSTM, Bayesian.	Matches PR-03 and the D2 literature (Aggarwal & Chugh 2022: competitive and more maintainable in constrained deployments); trivially re-trainable by future maintainers; the whole model is a table, so it can be shipped to the phone.

ID	Decision	Alternatives considered	Rationale / consequence
T7	**Routing in memory** with NetworkX A*, graph reloaded on admin edits.	`pgRouting` in the database; custom graph library.	Campus graph has fewer than 5 000 nodes; in-memory A* answers in milliseconds, is unit-testable without a database, supports per-request edge filtering (accessibility) without SQL changes, and free hosting plans often lack `pgRouting`.
T8	**Modular monolith** (one FastAPI process, strict package boundaries).	Micro-services per subsystem.	512 MB free-tier host, one deployable, no inter-service latency; subsystem independence (NFR-07) is preserved by interfaces, not by processes.
T9	**SVG floor plans + GeoJSON features** in Leaflet `CRS.Simple`.	Image tiles; canvas drawing; WebGL map.	Plans are vector in the source PDFs, remain sharp under zoom, are small, and can be pre-cached by the service worker (FR-10).
T10	**React Context + reducer** for the navigation session; TanStack Query for server state.	Redux; Zustand; MobX.	Session logic is a small finite-state machine (Figure 6) that is easiest to test as a pure reducer; server state benefits from query caching and retries.
T11	**PWA first, Capacitor Android APK second** (same code base).	Native Android app; Flutter; PWA only.	NFR-04 demands no installation for ordinary use; the APK is needed only for WiFi scanning and is built from the same React code.
T12	**Admin: local accounts with roles + JWT**, audit log for every mutation.	University SSO; HTTP basic auth.	SSO is out of scope; JWT with short expiry follows RFC 8725 recommendations; two roles keep a department coordinator away from graph and radio-map edits.
T13	**Model default `gpt-4o-mini`, configurable** (`OPENAI_MODEL`).	Fixed GPT-4.	Cost control (R03); intent extraction with structured output does not require the largest model; production can switch by configuration.
T14	**k-NN runs on the device** from a versioned radio map; the server builds, tests and publishes maps.	(a) k-NN on the server for every request (the D6 design); (b) both, chosen per request.	(a) is simpler (one code base) but sends every user's RSSI vector, i.e. their location, to the server every few seconds, and loads a free-tier server. We chose the device. **Cost:** the estimator exists twice (TypeScript on the client, Python on the server for evaluation); a parity test runs both on the same data and fails the build if results differ by more than 0.1 m. The radio map (about 0.3–0.6 MB per building, estimate) is public: the BSSIDs it lists are broadcast over the air anyway.
T15	**Dead reckoning between scans**: step detection + heading, blended with each WiFi fix and snapped to the corridor graph (priority SHOULD; MUST-level fallback below).	(a) Poll scans faster (not allowed by Android 9+: four scans per two minutes); (b) BLE beacons; (c) show only the last fix.	A fix every 30 s makes the blue dot jump through walls and stairwells. Steps and heading are available on every phone. Dead reckoning drifts, so each fix corrects it and map matching keeps it in corridors (Harle 2013; Newson & Krumm 2009). **Fallback if it proves unreliable:** the dot stays at the last fix with a growing uncertainty circle and the user taps "Next step"; navigation never depends on it.
T16	**LLM time budget 1.5 s, circuit breaker**, rules as fallback.	4 s timeout with retry; no LLM.	Worst case of the chain is 50 ms + 1 500 ms + 50 ms, plus database and composition, which stays below the 2 s requirement. After three consecutive failures the LLM is skipped for two minutes, so an outage costs nothing.
T17	**Origin is sent as a graph node id**, not as coordinates; the client holds the navigation graph.	Send (x, y, floor) with every request.	Data minimisation: a node id is only as precise as the corridor network. The cost is one more public download (`nav-graph`), which the client needs anyway for map matching and offline use.

1.3 Interface documentation guidelines

Every interface in Section 3 is documented in the same way so that implementers and testers can use it as a contract:

- **Signature** in Python 3.11 with full type hints (backend) or TypeScript 5 (frontend). Data transfer objects are Pydantic v2 models / TypeScript interfaces generated from the OpenAPI document.

- **Pre- and post-conditions** stated in prose, plus **errors** as typed exceptions mapped to the error codes in Section 3.2.
- **Units and conventions:** distances in metres, time in seconds (ISO 8601 UTC for timestamps, Europe/Istanbul for opening hours), RSSI in dBm, angles in degrees, floors as integers (basement negative, roof = 5), coordinates `(x, y)` in metres with origin at the bottom-left of the floor plan and the y-axis pointing up.
- **Naming:** `snake_case` for Python modules/functions and database columns, `PascalCase` for classes, `camelCase` in JSON and TypeScript, `SCREAMING_SNAKE` for error codes and constants, singular snake_case table names in the logical model, lower-case plural versioned API paths (`/api/v1/locations`).
- **Docstrings** follow the Google style; a public function without docstring, type hints or tests does not pass the continuous-integration gate (`ruff`, `mypy --strict` on `services/`, `eslint`, `tsc --noEmit`). TypeScript classes are documented with TSDoc.
- **Versioning:** REST resources live under `/api/v1`; breaking changes create `/api/v2`. The OpenAPI document is generated by FastAPI and published as `docs/openapi.json` (Swagger page at `/api/docs`); the frontend consumes generated types so a contract change fails the build.
- **Ownership:** each package has one owner who reviews every change to its public interface (Section 2.3); changes to the API contract need approval from the other two owners.
- **Language:** reports, code, commits and API names are English; user-facing strings are Turkish and English from i18n files. Reports are spell-checked before submission, as the syllabus requires.

1.4 Engineering standards

Standard / practice	Use in this project
UML 2.5.1 (OMG)	Class, sequence, state and package diagrams (Figures 1, 3–6); ER notation for Figure 2.
IEEE Std 1016-2009 (Software Design Descriptions)	Overall structure of this document: design viewpoints (decomposition, dependency, interface, data), design rationale (Section 1.2).
ISO/IEC/IEEE 29148:2018	Requirement identifiers (FR/NFR/PR) and the traceability matrix (Section 3.13).
ISO/IEC 25010:2011	Quality model for NFR coverage: performance efficiency, reliability, usability/accessibility, security, maintainability, portability.
OpenAPI 3.1, RFC 9110 (HTTP semantics)	REST contract, status codes, idempotency and caching headers (`ETag`, `Cache-Control`).
RFC 7519 (JWT) and RFC 8725 (JWT best practices)	Admin tokens: fixed algorithm, `exp`/`iat`/`sub`/`jti` claims, short lifetime, rotating refresh token.
OWASP ASVS 4.0 / OWASP Top 10 (2021) / OWASP Top 10 for LLM Applications	Security requirements and verification checklist (Section 3.11.1); prompt-injection handling.
WCAG 2.1 level AA (W3C)	Accessibility acceptance criteria for the client (Section 3.9.6).
KVKK (Law No. 6698) and ACM / IEEE-CS Code of Ethics	Data minimisation, explicit consent, no persistent position data (Section 3.11.2).
RFC 7946 (GeoJSON)	Feature format for floor plans. **Deviation:** RFC 7946 mandates WGS84; our files use a local metric CRS and are declared with a `crs`-less, project-specific profile documented in `docs/data-formats.md`.
IEEE 802.11 (RSSI definition)	RSSI is the received signal strength in dBm reported by the platform scan API; values are device-specific (Section 3.4.3).
PEP 8 / PEP 484, ESLint + Prettier	Code style and static typing, enforced in CI.
Semantic Versioning 2.0, Conventional Commits 1.0	Release and commit conventions (also feeds the per-person contribution record required by the course).

1.5 Design principles and how they are validated

The syllabus asks for the extent and validity of the design principles. For each principle the table states how it shapes the design and how we will check (not just claim) that it holds.

Principle	How it is applied	How it is validated
Layered architecture, separation of concerns	client → `api` → `services` → `repositories` → `db`; each layer calls only the layer below.	`import-linter` contract in CI fails the build on a skipped layer; code review checklist.
Single responsibility	The D4 `NLPEngine` was split into parser chain, resolver, validator and composer; positioning into providers, estimator, filter and map matcher.	Each class has one reason to change; reviewers reject classes that import from two unrelated packages.
Open/closed	New position sources, cost models and intent parsers are added as new classes behind `PositioningProvider`, `CostModel`, `IntentParser`.	Demonstrated in the project itself: `SimulatedProvider` and `AccessibleCost` were added without editing `PositionManager` or `RoutePlanner` (unit tests unchanged).
Dependency inversion, low coupling	Services depend on `Protocol` interfaces; collaborators are injected by FastAPI dependencies.	Unit tests replace repositories and the OpenAI client with fakes; no test needs a network.
Privacy by design	No position reaches the server; no table stores one; origin is a node id; logs are filtered.	Schema review; automated test that sends a request containing coordinates and greps the captured logs; data-flow table (Section 3.11.2).
Graceful degradation (fail-safe defaults)	Every external dependency has a fallback: LLM → rules; WiFi → QR → manual; network → offline cache; motion sensors → last fix + “Next step”.	Fault-injection runs: OpenAI key removed, network switched off, sensors denied; each must still answer or guide (Test Plan, E2E-OFF, E2E-NAV).
Zero hallucination (grounded AI)	The LLM only extracts intent and entities; every entity is looked up; answers come from templates.	Property test over random and adversarial strings: every id in a response exists in the database; evaluation set contains prompt-injection cases.
Accessibility by design	Step-free routing is a cost model, not a patch; every control is a native element with an accessible name.	axe-core and Lighthouse in CI; manual TalkBack/VoiceOver pass; WCAG checklist (Section 3.9.6).
Testability	Pure reducer for session state; pure functions for routing and k-NN; deterministic seeds and a simulated walk.	Coverage targets (backend ≥ 80 %, frontend ≥ 70 %); table-driven tests; Section 3.14.

1.6 Design patterns

Identifying patterns is central to object design (Bruegge & Dutoit, 2004; Gamma et al., 1994).

Pattern	Where	Purpose
Strategy	`PositioningProvider`, `CostModel`, `IntentParser`	Interchangeable algorithms behind a stable interface.
Chain of Responsibility	`HybridIntentParser`	Try fast path, cache, LLM, rules in order; first confident result wins.
Facade	`NavigationOrchestrator`, `PositionManager`, `RoutePlanner`	One entry point that coordinates several collaborators for its caller.
Repository	`CampusRepository`, `NavRepository`, `RadioMapRepository`, `AdminRepository`	Hide SQL/PostGIS details; services are testable with in-memory fakes.
Adapter	`OpenAIIntentParser`, `WifiScanner` plug-in bridge, Web Speech wrappers	Isolate vendor APIs; swap or fake them in tests.
Observer	`PositionManager.subscribe()`; `GraphInvalidated`, `RadioMapPublished` events	Map, navigation and voice update when the position changes; in-memory graph and maps reload after admin edits.

Pattern	Where	Purpose
Singleton (application scope)	`NavGraph`, `RadioMapRegistry`	Created once in the FastAPI lifespan handler and injected, not global variables.
State	Client `sessionReducer` / `NavigationController`	Explicit navigation-session states and legal transitions (Figure 6).
Decorator	`CachingParser`, `TimeoutParser`, circuit breaker	Add caching, time limits and failure isolation to any parser without changing it.
Template Method	`InstructionGenerator` (segment → text templates)	Same step logic for Turkish and English wording.

1.7 Creativity of the design

We kept the system simple enough for three people on free hosting, but went beyond a “map and search box” in five places:

- **Privacy-friendly positioning.** The position is computed on the phone from a downloaded radio map, so even a compromised server cannot reveal where users have been. Because the server only receives a corridor node id when a route is requested, “no position is stored” is verifiable from the architecture instead of being a policy sentence.
- **Continuous blue dot despite Android’s scan limit.** Between scans (every 30 s) the app moves the dot with detected steps and compass heading, corrects it with every WiFi or QR fix, and snaps it to the nearest corridor so it never appears inside a wall.
- **QR anchors as a universal backup.** Stickers at stairs and corridor crossings give an exact position on every phone, including iPhones, at no hardware cost. (Future work, not in the MUST scope: with separate consent, a scan at a QR anchor could contribute one anonymous fingerprint to refresh the radio map; contributions would be reviewed by an admin before training.)
- **Provably grounded AI.** “Rules first, AI second”: simple questions such as “Where is D232?” are answered in milliseconds without the LLM; the LLM is an optimisation, never a dependency, and it can never put a non-existent room on the screen.
- **Explainable accessibility routing and data from existing PDFs.** The response states why a route avoids stairs or reports precisely that no step-free route exists; floor plans, room names and anchors are extracted automatically from the university’s own vector wayfinding PDFs, so the same pipeline digitises future buildings.

1.8 New tools and technologies

The following were new to the team and are learned during CMPE 492 (lifelong learning). Each has a reason and a risk-reduction step.

Tool / technology	Used for	How we reduce the learning risk
Capacitor + a Kotlin plug-in	WiFi scanning on Android from the same React code base	Spike in week 1–2; the data-collection app is the first consumer, so the plug-in is exercised early.
PostGIS, `pg_trgm`	Spatial queries, typo-tolerant Turkish search	Seed script and migrations from week 1; unit tests for folding.
OpenAI structured outputs	Safe JSON intent extraction	Schema generated from the Pydantic model; mocked in CI.
NetworkX A*	Routing with per-request edge filtering	Hypothesis property test against Dijkstra.
scikit-learn `GridSearchCV` **+** `GroupKFold`	Honest accuracy estimates for k-NN	Evaluation script reproducible from one command.
Workbox, IndexedDB, Web Speech, Sensor APIs	Offline mode, voice, step detection	Each wrapped by an adapter with a fake for tests.
`openapi-typescript`, `import-linter`	Contract-first API, enforced layering	Both run in CI from the first pull request.

1.9 Definitions, acronyms and abbreviations

See Section 4 (Glossary) for the complete list. The following abbreviations are used throughout: **AP** access point; **BSSID** MAC address of an AP radio; **RSSI** received signal strength indicator (dBm); **k-NN** k-nearest neighbours; **PDR** pedestrian dead reckoning (steps + heading); **LLM** large language model; **DTO** data transfer object; **FSM** finite-state machine; **PWA** progressive web app; **CRS** coordinate reference system; **SRID** spatial reference identifier; **ETA** estimated time of arrival; **MoSCoW** must/should/could/won't prioritisation.

2. Packages

2.1 Layer and package overview

Figure 1 shows the three layers inherited from D6 and the packages inside them. Arrows point in the direction of allowed dependencies: the client talks to the server only through the versioned REST API; services depend on repositories only through interfaces; nothing depends on the API package. The client's `services/positioning` package (highlighted) never sends data to the network; its only server traffic is downloading the public radio map and navigation graph.

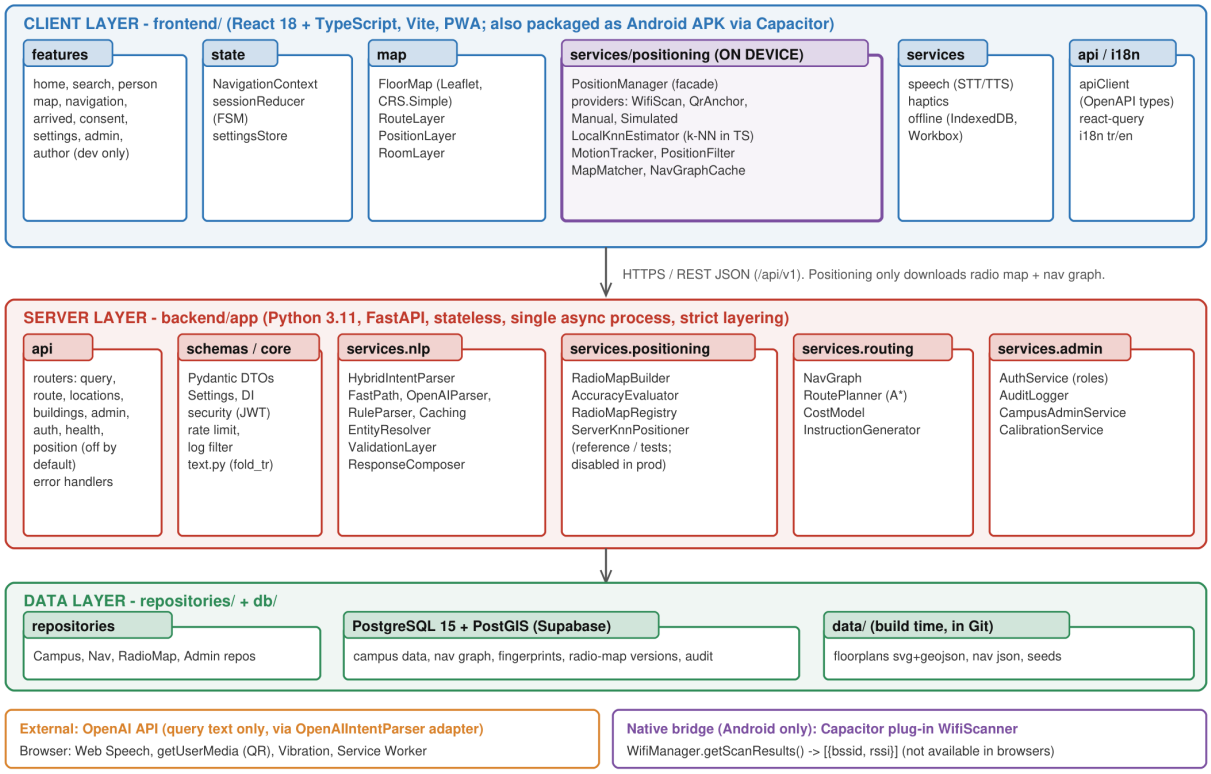


Figure 1 – Package and layer view

2.2 Repository layout

```
tedu-guide/
  backend/app/  main.py  core/  api/  schemas/  db/  repositories/  services/{nlp,positioning,routing,admin}/
        tests/
  frontend/src/ app/  features/{home,search,person,map,navigation,arrived,admin,consent,settings,author}/
                components/  map/  services/{positioning,speech,haptics,offline}/  state/  api/  i18n/  types/
  collector/    Capacitor plug-in WifiScanner (Kotlin) and the fingerprint-collection screen
  data/         floorplans/<block>/<floor>.{svg,rooms.json,geojson}  nav/<block>.json  seeds/  fingerprints/
        eval/
  tools/        pdf_to_svg.py  extract_rooms.py  build_seed.py  graph_check.py  train_knn.py  simulate_walk.py
        create admin.py
  docs/         api-contract.md  openapi.json  data-formats.md  LLD / Test Plan / Backlog
```

2.3 Package ownership and dependencies

Package	Responsibility	Depends on	Owner
`app.api`	HTTP routers, request validation, error-to-status mapping; no business logic.	`schemas`, `services`, `core`	Taci
`app.core`	Settings, dependency injection, security helpers, rate limiting, structured logging, log filter, exception types, `fold_tr`.	—	Taci

Package	Responsibility	Depends on	Owner
`app.schemas`	Pydantic DTOs shared with OpenAPI.	—	Taci
`app.db`, `app.repositories`	SQLAlchemy models, sessions, Alembic migrations, repositories (campus, navigation, radio maps, admin).	PostgreSQL/PostGIS	Taci
`services.routing`	`NavGraph`, `RoutePlanner`, `CostModel`, `InstructionGenerator`, public graph export.	`repositories`	Taci
`services.admin`	Authentication, roles, token handling, audit log, campus administration use cases, `CalibrationService`.	`repositories`, `core`, `services.positioning`	Taci
`services.nlp`	Normalisation, intent parsers, entity resolution, validation layer, response composer.	`repositories`, OpenAI SDK	Elif
`services.positioning` (server)	`RadioMapBuilder`, `AccuracyEvaluator`, `RadioMapRegistry`, reference `ServerKnnPositioner`.	`repositories`, scikit-learn	Elif (data, model, evaluation); Taci (`RadioMapRegistry`, endpoints)
`frontend/services/positioning` (device)	Providers, `LocalKnnEstimator`, `MotionTracker`, `PositionFilter`, `MapMatcher`, `NavGraphCache`, `PositionManager`.	browser APIs, Capacitor, `offline`	Elif (estimator, filter, scan plug-in) with Fatima (UI glue, QR, map matcher)
`collector`	Android WiFi scanning plug-in and data-collection UI.	Capacitor, Android SDK	Elif (+ Fatima for UI glue)
`frontend/features`, `components`, `map`	Screens, Leaflet floor-plan viewer, route overlay, accessibility.	`api`, `state`, `services`	Fatima
`frontend/services` (speech, haptics, offline)	Speech services, vibration, offline cache.	browser APIs, Capacitor	Fatima (speech services joint with Elif)
`data`, `tools`	Floor plans, navigation graphs, seed data, extraction tools.	PyMuPDF, shapely	Fatima (+ Taci for `graph_check`, `build_seed`)

Dependency rules (enforced by an import-linter configuration in CI): `api → services → repositories → db`; `services` never import `api`; `services.nlp` and `services.positioning` do not import each other (they meet only in `NavigationOrchestrator` and in the API layer); frontend `features` never call `fetch` directly, only the generated API client; the client's `positioning` package imports no HTTP code except the two download calls in `OfflineStore`.

2.4 Deployment view

Node	Technology	Artifacts	Notes
Browser / PWA	Chrome, Safari, Firefox (mobile + desktop)	Static bundle, service worker, SVG plans	No installation (NFR-04). Manual and QR positioning only.
Android device	Capacitor APK (side-loaded)	Same bundle + `WifiScanner` plug-in	WiFi positioning, step detection, data collection.
CDN / static host	Vercel	`frontend/dist`	SPA rewrite, `VITE_API_URL`.
API host	Render or Railway (Docker)	`backend` image	One async Uvicorn process; health check `/health`; cold-start mitigation by periodic ping.
Database	Supabase / Railway PostgreSQL 15 + PostGIS	Schema, seed data, published radio maps	Free-tier project must be kept active (weekly keep-alive).
External	OpenAI API	—	Receives the query text only.

2.5 Configuration

All settings come from environment variables read once by `pydantic-settings`; a wrong or missing value stops the start-up. Only `.env.example` is committed.

Variable	Meaning	Default / note
`DATABASE_URL`	Async PostgreSQL URL (`postgresql+asyncpg://…`)	required
`JWT_SECRET`, `ACCESS_TOKEN_MINUTES`, `REFRESH_TOKEN_DAYS`	Admin token signing key and lifetimes	secret required; 15 min; 7 days
`CORS_ORIGINS`	Comma-separated allow-list of front-end origins	dev: `http://localhost:5173`
`OPENAI_API_KEY`, `OPENAI_MODEL`, `LLM_TIMEOUT_S`	Key (optional), model name, hard time limit	key empty = rules only; `gpt-4o-mini`; 1.5
`LLM_BREAKER_FAILURES`, `LLM_BREAKER_PAUSE_S`	Circuit breaker thresholds	3; 120
`ENABLE_SERVER_POSITIONING`	Mounts the reference `POST /position` endpoint	false in production, true in tests and field evaluation
`ADMIN_SEED_USER`, `ADMIN_SEED_PASSWORD`	Creates the first admin when none exists	unset in production after first start
`RATE_LIMIT_QUERY`, `RATE_LIMIT_LOGIN`	Requests per minute and IP	60; 5

3. Class Interfaces

This section describes, component by component, what each class is responsible for and which methods it offers to other classes. Simple presentational React components are not listed. Section 3.1 (data) and 3.2 (REST contract) come first because every other component depends on them.

3.1 Data model (SS-07)

Figure 2 shows the logical model. All geometry columns use SRID 0 (Cartesian metres). `search_text` columns hold the **application-folded** form of code, name and aliases (Section 3.3.1) because the PostgreSQL `unaccent` dictionary does not handle the Turkish dotted/dotless i consistently.

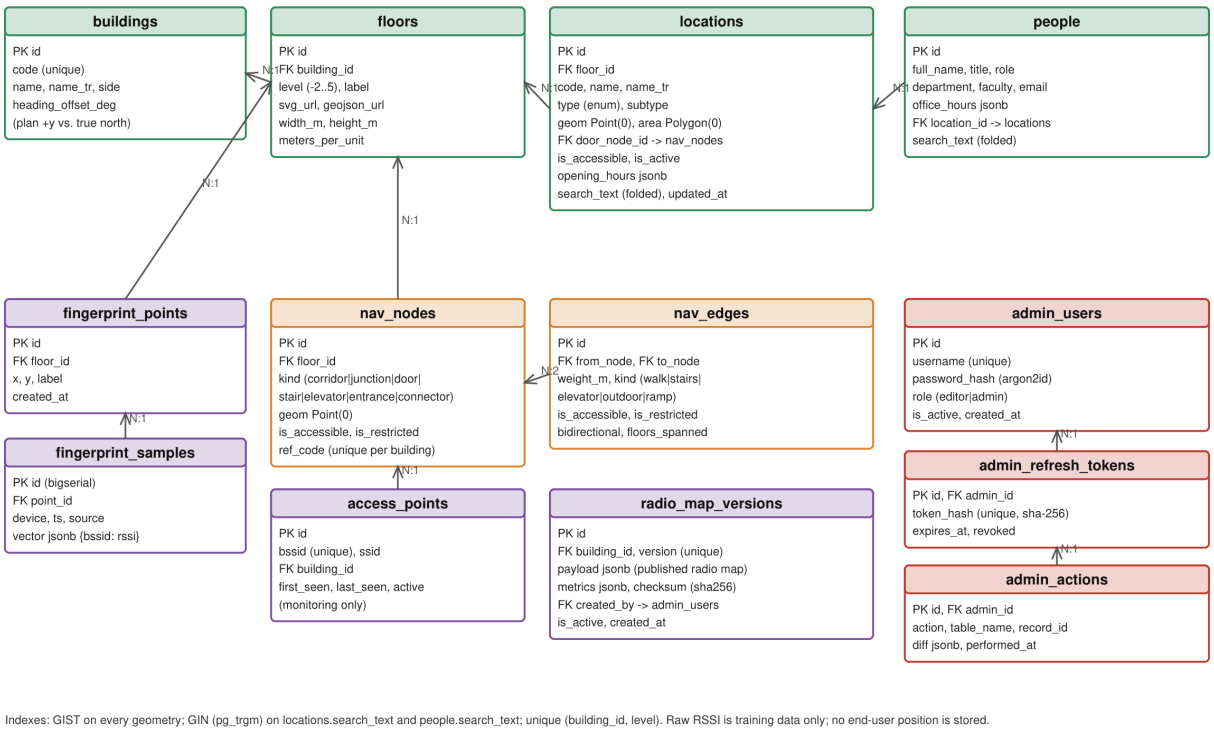


Figure 2 – Logical data model

```
buildings(id, code UNIQUE, name, name tr, side, color, heading offset deg)
floors(id, building id FK, level INT, label, svg url, geojson url, width m, height m, meters per unit,
      UNIQUE(building id, level))
locations(id, floor id FK, code, name, name tr, type, subtype, geom geometry(Point,0), area geometry(Polygon,0)
      NULL,
           door_node_id FK NULL, is_accessible, is_active, opening_hours JSONB, search_text, updated_at)
people(id, full name, title, role, department, faculty, email, office hours JSONB, specialization, location id
      FK NULL, search_text)
nav nodes(id, floor id FK, kind, geom geometry(Point,0), is accessible, is restricted, ref code)   -- ref code
      unique per building
nav_edges(id, from_node FK, to_node FK, weight_m, kind, is_accessible, is_restricted, bidirectional,
      floors spanned)
fingerprint points(id, floor id FK, x, y, label)   fingerprint samples(id, point id FK, device, ts, source,
      vector JSONB)
access points(id, bssid UNIQUE, ssid, building id FK, first seen, last seen, active)           -- monitoring
      only
radio map versions(id, building id FK, version, payload JSONB, metrics JSONB, checksum, created by FK,
      is_active, created_at)
admin users(id, username UNIQUE, password hash, role, is active, created at)
admin refresh tokens(id, admin id FK, token hash UNIQUE, expires at, revoked)
admin_actions(id, admin_id FK, action, table_name, record_id, diff JSONB, performed_at)
-- indexes: GIST on every geometry; GIN (pg_trgm) on locations.search_text, people.search_text;
      UNIQUE(building_id, level), UNIQUE(building_id, version)
```

Column	Allowed values / meaning
`locations.type`	`classroom`, `office`, `lab`, `amphi`, `studio`, `meeting`, `facility`, `wc`, `circulation` (corridors, lobbies, stairs/lift labels shown on the plan).
`locations.subtype`	Only for `facility`: `library`, `cafeteria`, `refectory`, `prayer`, `health`, `print`, `sports`, `baby_care`, `dean_office`, … Used by “nearest facility” queries.
`nav_nodes.kind`	`corridor`, `junction`, `door`, `stair`, `elevator`, `entrance`, `connector`.

Column	Allowed values / meaning
nav_edges.kind	walk, ramp, stairs, elevator, outdoor.
admin_users.role	editor (locations, opening hours, people), admin (everything, including graph and radio maps).
fingerprint_samples.source	survey (team phones), crowd (reserved for the optional future feature, Section 1.7).

Integrity rules. (1) Every location of type classroom, office, lab, amphi, studio, meeting, facility has a door_node_id; (2) edges connect nodes of the same floor unless kind is stairs or elevator (then floors_spanned ≥ 1) or outdoor; (3) fingerprint_points.(x,y) lies inside the floor extent; (4) deleting a location that is referenced by a person is rejected; (5) nav_nodes.ref_code is unique within a building (it is the target of QR payloads); (6) at most one radio_map_versions row per building is is_active. Rules (1)–(3) and (5) are verified by tools/graph_check.py in CI and again at application start-up.

Migrations and seed. Alembic with the async template; migration 0001 creates the postgis and pg_trgm extensions. Seed data is idempotent (INSERT … ON CONFLICT DO UPDATE), generated by tools/build_seed.py from data/ so that the database can be rebuilt from the repository alone. No table stores the location of a user.

3.2 REST API contract (SS-02)

All endpoints are under /api/v1, exchange UTF-8 JSON and are documented by the generated OpenAPI document. Unauthenticated access is allowed for everything except /admin/* (FR-12). Requests that may carry an origin use POST so that nothing about the user's whereabouts appears in URLs or access logs.

Method + path	Purpose	Request (essential fields)	Success response	Errors
GET /health	Liveness / readiness	—	{status, version, db, radioMaps}	503
POST /query	Natural-language query (FR-03)	text, lang?, accessible?, origin?	{intent, entities, parsedBy, verified, answer{text,textTr}, results[], clarification?, route?, latencyMs}	422, 429
POST /route	Route between two points (FR-04/05)	from{nodeId | locationId | position}, to{locationId}, accessible, lang?	Route (steps, polylines, GeoJSON, distance, ETA)	404 ROUTE_NOT_FOUND, 409 NO_ACCESSIBLE_ROUTE, 422 POSITION_OFF_GRAPH
GET /locations/search	Search places and people (FR-07)	q, type?, building?, limit?	Location[] with occupants	422
GET /locations/{id}, /people/{id}	Detail	—	Location / Person	404
GET /buildings, /buildings/{code}/floors	Floor-plan metadata (FR-06)	—	[{level,label,svgUrl,geojsonUrl,widthM,heightM,metersPerUnit}]	404
GET /buildings/{code}/nav-graph	Public navigation graph for map matching and node lookup	If-None-Match	{version, nodes[], edges[]} (restricted areas omitted); ETag	304, 404
GET /buildings/{code}/radio-map	Active radio map for on-device k-NN	If-None-Match	RadioMap JSON (Section 3.4.1); ETag	304, 404 RADIO_MAP_NOT_FOUND

Method + path	Purpose	Request (essential fields)	Success response	Errors
POST `/position`	Reference server-side estimate (tests, field evaluation); **off by default**	`method: wifi|qr|manual`, `rssi[]` / `qrPayload` / `manual`	`{position, confidence, errorEstimateM, method, fallbackRequired}`	404 when disabled, 422, 503 `MODEL_NOT_LOADED`
POST `/admin/login`, `/admin/refresh`	Authentication (FR-08)	`username,password` / `refreshToken`	`{accessToken, refreshToken, expiresIn, role}`	401, 429
`/admin/locations`, `/admin/people`, `/admin/hours/{id}` (GET/POST/PUT/DELETE)	Campus data administration (role `editor` or `admin`)	Resource JSON + Bearer token	Updated resource	401, 403, 404, 409
POST `/admin/graph/import`	Replace a building's navigation graph from `nav/<block>.json` (role `admin`)	JSON upload	`{nodes, edges, warnings[]}`; publishes `GraphInvalidated`	401, 403, 422
POST `/admin/fingerprints/import`, `/admin/recalibrate`	Import survey CSV; build, evaluate and publish a radio map (UC-07, role `admin`)	CSV upload / `{building, force?}`	`{status, version, metrics{medianM, p90M, floorAcc}, published}`	401, 403, 422
GET `/admin/radio-maps`, POST `/admin/radio-maps/{version}/activate`	List versions; roll back or activate one	`building`	`RadioMapVersion[]` / `{active}`	401, 403, 404
GET `/admin/audit`	Audit trail	`from?, to?, adminId?`	`AuditEntry[]`	401, 403

Origin. `origin` and `from` accept one of three forms: `{"nodeId": 1234}` (preferred: the client resolves its position to a graph node on the device, T17), `{"locationId": 412}` ("I am at room …" from the manual picker) or `{"buildingCode":"D","floor":0,"x":41.2,"y":6.8}` (accepted for clients without a graph; snapped by the server and never stored). None of the three is written to a table or a log.

Example – query request and response

```
POST /api/v1/query
{ "text": "D232 nerede?", "accessible": false, "origin": {"nodeId": 8841} }

200 OK
{ "intent": "locate", "entities": {"room":"D232","person":null,"facilityType":null,"floor":null},
  "parsedBy": "fastpath", "verified": true,
  "answer": {"text": "D232 is Classroom D232, Block D, floor 2.", "textTr": "D232, D Blok 2. kattaki D232
        numaralı sınıftır."},
  "results": [{"locationId":412,"code":"D232","name":"Classroom D232","type":"classroom",
               "building":"D","floor":2,"x":57.9,"y":21.4,"accessible":true,"score":1.0}],
  "clarification": null, "route": null, "latencyMs": 38 }
```

Example – radio map (client format)

```
GET /api/v1/buildings/D/radio-map            ->  200, ETag: "D-2026-11-03-1", Cache-Control: no-cache
{ "version": "D-2026-11-03-1", "building": "D", "featureMode": "raw", "metric": "euclidean",
  "missingDbm": -100, "floorK": 5, "posK": 3, "tauDb": 15, "minKnownAps": 3,
  "aps": ["aa:bb:cc:00:11:22", "aa:bb:cc:00:11:23", "..."],              // ordered AP universe
  "points": [ {"f": 0, "x": 12.4, "y": 3.1, "v": [-61, -100, -74, "..."]}, "..." ],   // one row per reference
        point
  "metrics": {"medianM": 2.7, "p90M": 4.6, "floorAcc": 0.96},   // example values, not measurements
  "checksum": "sha256:..." }
```

Caching and errors. Plans, navigation graphs and radio maps carry an `ETag`; clients revalidate with `If-None-Match`. Every error response has the shape `{"error":{"code":"ROUTE_NOT_FOUND","message":"…","details":{}}}` and the header `X-Request-ID`. Messages are human-readable English; the client maps `code` to a localised string.

Code	HTTP	Meaning / client behaviour
VALIDATION_ERROR	422	Schema violation; client shows the field message.

Code	HTTP	Meaning / client behaviour
UNAUTHENTICATED / FORBIDDEN	401 / 403	Missing or expired token (client refreshes once, then logs out) / role too low.
NOT_FOUND	404	Unknown id.
ROUTE_NOT_FOUND	404	Start and goal are not connected in the graph; client offers manual start or another destination.
NO_ACCESSIBLE_ROUTE	409	A route exists only through non-accessible edges; client states this explicitly (FR-05) and offers the standard route as a clearly labelled alternative.
POSITION_OFF_GRAPH	422	Origin cannot be snapped to a nearby node; client asks for manual pin-drop.
RADIO_MAP_NOT_FOUND	404	No active radio map for this building; client disables the WiFi provider and uses QR/manual.
MODEL_NOT_LOADED	503	Reference server estimator has no radio map (only when `ENABLE_SERVER_POSITIONING` is on).
RATE_LIMITED	429	Back-off using `Retry-After`.
INTERNAL_ERROR	500	Unexpected; request id shown for support.

An AI outage is **not** an error: the orchestrator falls back to the rule parser and sets `parsedBy: "rules"` so the user still receives a verified answer. **Rate limits** (per IP, per minute): `/query` 60, `/route` 60, `/admin/login` 5, all other public reads 120.

3.3 Natural-language subsystem (SS-03)

Responsibility. Convert free-form Turkish/English text into a structured, **database-verified** query result and a templated answer within the 2-second budget (NFR-01), without ever returning a place or person that does not exist in the campus database (T5). **Owner:** Elif. **Requirements:** FR-03, FR-07, PR-02, NFR-01, NFR-06.

3.3.1 Text normalisation

`fold_tr(text)` is the single normalisation function used for queries and for the stored `search_text`. It (1) applies Unicode NFKC, (2) maps `İ→i` and `I→ı` **before** lower-casing (Python's default `lower()` turns `I` into `i`, which breaks Turkish), (3) folds `ç→c, ğ→g, ı→i, ö→o, ş→s, ü→u`, (4) collapses punctuation and whitespace. `normalize_room_code(text)` recognises codes such as `d 232`, `D-232`, `d232.` with the pattern `\b([A-HKR])\s*[-.]?\s*(\d{3})\b` and returns `D232` (all block letters A–H, K and the Rectorate `R`). Codes that do not follow the block-plus-three-digits pattern (for example `A-B013`) are matched through the folded code index of the database, not through the regular expression. Required unit tests: "Çapın / capin / CAPIN", "İbadethane / ibadethane", "Kütüphane / kutuphane / library", "h 101", "a-b013" (the prototype fails the first and third).

3.3.2 Parser interfaces

Intent parser chain

```
class Intent(StrEnum): NAVIGATE; LOCATE; QUERY_HOURS; FIND_PERSON; GENERAL_INFO; UNKNOWN

@dataclass(frozen=True)
class Entities:  room: str|None; person: str|None; facility type: str|None; floor: int|None; building: str|None

@dataclass(frozen=True)
class ParsedQuery:
    intent: Intent; entities: Entities; language: Literal['tr','en','unknown']
    wants_accessible: bool; confidence: float          # 0..1
    parsed by: Literal['fastpath','cache','llm','rules']

class IntentParser(Protocol):
    timeout s: float
    async def parse(self, text: str, lang hint: str | None = None) -> ParsedQuery | None: ...
    # None = 'cannot handle, ask the next parser'.  Never raises for bad input.
```

"Nearest toilet" or "en yakın kafeterya" is a `LOCATE` or `NAVIGATE` intent with `facility_type` set; the resolver then uses the origin node (if supplied) to find the nearest matching location.

Class	Behaviour	Time limit
`FastPathParser`	Matches normalised room codes and exact place names against an in-memory index of all `locations`; patterns such as “where is X”, “X nerede”, “X’ye git”, bare “D232”. Returns `None` when the sentence contains more words than the pattern allows. Cost < 5 ms.	50 ms
`CachingParser(inner)`	Decorator. LRU cache of 1 000 entries, TTL 1 h, key = `fold_tr(text)`; stores only `ParsedQuery`, never user identifiers or origin.	—
`OpenAIIntentParser`	Adapter around `AsyncOpenAI`. Structured output with a JSON schema generated from `ParsedQuery`, `temperature=0`, `max_tokens=200`. The system prompt restricts the model to extraction (“never invent rooms or people”) and contains 10 few-shot examples in both languages. The user text is wrapped in `<query>…</query>` and treated as data (prompt-injection mitigation). Only the text is sent: no origin, no identifiers. **No retry** (a retry would break the budget). A `CircuitBreaker` skips the LLM for 120 s after 3 consecutive failures or timeouts.	1.5 s
`RuleIntentParser`	Deterministic parser (corrected port of the prototype’s `parseQuery`): token-level person matching (not substring), Turkish patterns (`nerede`, `nasıl giderim`, `kaçta`, `hocanın odası`), room-code regex. Always returns a `ParsedQuery` (intent `UNKNOWN` at worst).	50 ms
`HybridIntentParser(chain)`	Chain of Responsibility: `FastPath → Cache(LLM) → Rules`. Each parser runs under `asyncio.wait_for`; exceptions and timeouts are logged (without the text) and the next parser is tried. Emits metrics: path distribution, p50/p95 latency, LLM failure rate.	≤ 1.6 s worst case

3.3.3 Entity resolution, validation and answer composition

EntityResolver / ValidationLayer / ResponseComposer

```
class Resolution(BaseModel):
    kind: Literal['exact','ambiguous','none']
    locations: list[Location]; people: list[Person]; suggestions: list[Location|Person]

class EntityResolver:
    def __init__(self, campus: CampusRepository, min_accept: float = 0.88, min_suggest: float = 0.70): ...
    async def resolve(self, e: Entities, origin_node: NodeId | None = None) -> Resolution

class ValidationLayer:
    async def validate(self, parsed: ParsedQuery, res: Resolution) -> VerifiedResult
    # postcondition: every id in VerifiedResult exists in the database at call time;
    #                free text produced by the LLM is never copied into the result.

class ResponseComposer:                                  # templates (tr, en) + values from the database
    def compose(self, result: VerifiedResult, lang: str, now: datetime) -> Answer   # Answer(text, text_tr)
```

Algorithm. (1) Room: exact `code` match = score 1.0; otherwise `rapidfuzz.token_set_ratio` on `search_text`, divided by 100. (2) Person: token-set match on folded full name (whole tokens, never substrings — the prototype returns “Yağmur Ar” for “library” because “ar” is a substring). (3) Facility type: the nearest accessible-compatible location of that `subtype` to the origin node, measured on the navigation graph when an origin is supplied, otherwise alphabetical. (4) Score ≥ `min_accept` and the runner-up is at least 0.05 lower → `exact`; two or more candidates within 0.05 or scores between `min_suggest` and `min_accept` → `ambiguous` (at most three options are shown); otherwise `none` with up to three suggestions. The thresholds are configuration values tuned on the evaluation set.

Answers. `ResponseComposer` fills fixed templates, for example “The library is open today until 22:00.” (`QUERY_HOURS`, using `OpeningHours`, Section 3.6.1), “Prof. Çapın’s office is D207, Block D, floor 2.” (`FIND_PERSON`), “D232 is Classroom D232, Block D, floor 2.” (`LOCATE`). A wrong LLM output can therefore at most lead to a “not found” message.

Evaluation (planned, T-73). `data/eval/nlp_queries.csv`: at least 150 labelled queries (Turkish and English, typos, ambiguous, out-of-scope, non-existent rooms such as `D999`, prompt-injection attempts). Acceptance: intent accuracy ≥ 95 %, entity F1 reported, **hallucination rate = 0** (property test: no returned id outside the database), end-to-end p95 latency < 2 s with the fast path enabled.

3.4 Positioning subsystem (SS-04)

Responsibility. Produce a position `(building, floor, x, y)` with a confidence value from whichever signal is available, keep it moving smoothly between measurements, and never block the rest of the system when no automatic signal exists. Most classes run **on the device**; only the classes that build, test and publish radio maps run on the server (T14). **Owner:** Elif (data, model, estimator, collector, plug-in) with Taci (`RadioMapRegistry`, endpoints) and Fatima (QR, map matcher, UI glue). **Requirements:** FR-01, FR-02, PR-03, NFR-05, NFR-06.

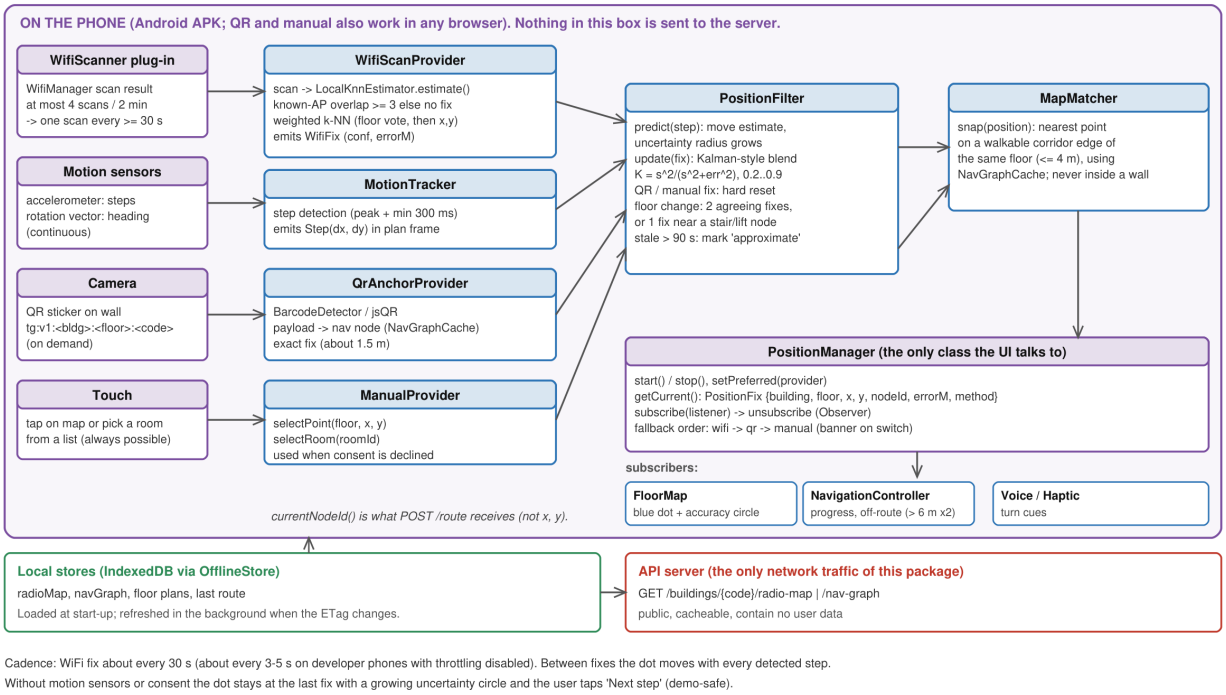


Figure 3 – Positioning pipeline on the device

3.4.1 Radio map format and delivery

A radio map is one JSON document per building (example in Section 3.2). `aps` is the ordered AP universe; each row of `points` is a reference point with floor, plan coordinates and the RSSI vector in that order (missing AP = `missingDbm`, integers only to keep the file small). `featureMode`, `metric`, `floorK`, `posK`, `tauDb` and `minKnownAps` are chosen by the training run so that the TypeScript estimator needs no hard-coded constants. **Size estimate:** about 500 points × 150 APs ≈ 75 000 integers, i.e. 0.3–0.6 MB as JSON before compression (to be measured). The client downloads the active map with `If-None-Match` at start-up and once a day while online, stores it in IndexedDB and keeps the previous version if a download fails. The Settings screen shows the map version.

3.4.2 Provider interface and manager (device, TypeScript)

PositionProvider, PositionManager

```
export interface PositionFix { buildingCode: string; floor: number; x: number; y: number; nodeId: number |
null;
  confidence: number; errorM: number; method: 'wifi'|'qr'|'manual'|'simulated'|'dead-reckoning'; approximate:
       boolean; ts: number }
export interface PositionProvider {                          // Strategy
  readonly id: 'wifi'|'qr'|'manual'|'simulated';
  isAvailable(): Promise<boolean>;                            // false in a normal browser for 'wifi'
  start(onFix: (f: PositionFix) => void, onError: (e: PositioningError) => void): void;
  stop(): void;
}
export class PositionManager {                               // Facade + Observer; the only class the UI talks
to
  constructor(providers: PositionProvider[], motion: MotionTracker, filter: PositionFilter, matcher:
       MapMatcher);
  start(): void; stop(): void; getCurrent(): PositionFix | null;
  subscribe(listener: (f: PositionFix) => void): () => void;    // returns the unsubscribe function
  setPreferred(id: PositionProvider['id']): void;
  selectPoint(floor: number, x: number, y: number): void;       // delegates to ManualProvider
  selectRoom(roomId: number): void;
}
```

```
export class LocalKnnEstimator {
  loadRadioMap(map: RadioMap): void;
  estimate(scan: ScanResult[]): WifiFix | null;              // null if fewer than minKnownAps known APs
}
export class MotionTracker  { start(): void; stop(): void; onStep(cb: (s: Step) => void): () => void }
export class PositionFilter { predict(step: Step): void; update(fix: PositionFix): void; state(): FilterState }
export class MapMatcher     { snap(p: Point2D, floor: number, headingDeg?: number): { point: Point2D; nodeId:
      number | null; snapped: boolean } }
export class NavGraphCache  { load(buildingCode: string): Promise<void>; nearestNode(floor: number, x: number,
      y: number): number | null;
                              edgesOnFloor(floor: number): Edge[]; nodeById(id: number): GraphNode }
```

Provider	Input	Method	Expected error	Platforms
`WifiScanProvider`	`WifiScanner.scan()` result: `[{bssid, ssid, rssi, frequency}]`	Scan about every 30 s (every 3–5 s when throttling is switched off on a developer phone); `LocalKnnEstimator.estimate()`.	Target median ≤ 3 m, p90 ≤ 5 m (to be measured)	Android APK
`QrAnchorProvider`	Camera frame → payload `tg:v1:<building>:<floor>:<code>`	`code` is the `ref_code` of a navigation node (stairs, corridor crossing) or a location code; the node is looked up in `NavGraphCache`. No secret is needed: a forged QR only mis-locates its own holder.	≈ 1.5 m	All devices with a camera
`ManualProvider`	Point on the map or a place picked from the list	Validates that the point lies on the floor extent; the nearest node is attached.	User dependent	All
`SimulatedProvider`	Recorded walk (`tools/simulate_walk.py`)	Replays timestamped fixes.	—	Demo, tests

Selection logic. User preference → `wifi` if `Capacitor.getPlatform() === 'android'`, the plug-in is available, a radio map is loaded and consent was given → `qr` → `manual`. When `LocalKnnEstimator` returns `null` or low confidence (fewer than three known APs or confidence < 0.25), `PositionManager` shows a non-blocking banner (“Weak WiFi map here – scan a QR sticker or tap the map”) and keeps using the filter prediction until a QR/manual fix arrives. Listeners are notified when the position changes by at least 0.5 m, the floor changes, or the approximate flag flips, at most five times per second.

3.4.3 k-NN estimator (device)

Step	Specification
1 Vector	For a scan, build a vector over `aps`: observed RSSI, missing AP = `missingDbm` (−100), values below −90 clipped to −100 (the same preprocessing as training). `n_known` = number of scanned BSSIDs that are in `aps`.
2 Guard	If `n_known < minKnownAps` (3) return `null`.
3 Distances	Distance (metric from the map, default Euclidean in dB) to every reference point: O(points × APs) ≈ 75 000 operations, a few milliseconds.
4 Floor	The `floorK` (5) nearest points vote for the floor with weight `1/(d + ε)`; the floor with the largest total weight wins. Wrong-floor errors are far worse than wrong-metre errors, hence the separate stage.
5 Position	Among the points of that floor take the `posK` (3) nearest; position = weighted mean of their coordinates (weights `1/(d + ε)`).
6 Confidence	`conf = exp(−d̄ / τ) × min(1, n_known / 6)` where $\bar{d}$ is the mean distance to the `posK` neighbours and τ = `tauDb` (15 dB, tuned). `errorM` = distance-weighted spread of the neighbours’ coordinates, at least 1.5 m.

3.4.4 Fusion between scans: motion, filter and map matching

These classes are priority SHOULD (T15); everything above works without them.

Class	Specification
`MotionTracker`	Steps from the accelerometer: low-pass filtered magnitude, adaptive peak threshold (mean + 0.8 σ over 2 s), at least 300 ms between steps. Heading from the rotation-vector sensor (absolute orientation), converted to the plan frame with `buildings.heading_offset_deg`; samples are ignored while the sensor reports low accuracy. Step length starts at 0.65 m and is corrected slowly (`L ← 0.9 L + 0.1 · distance between two good fixes / steps between them`, clamped to 0.5–0.9 m). Assumption: the phone is held in the hand while navigating.
`PositionFilter`	State `(floor, x, y, σ).predict(step)` moves the estimate by one step length along the heading and increases σ by 0.15 m per step (to be tuned). `update(fix)` uses a scalar gain `K = σ² / (σ² + errorM²)` clamped to 0.2–0.9 and sets `σ ← sqrt((1 − K) σ²)`. A QR fix resets σ to 1.5 m, a manual fix to 3 m. **Floor change** is accepted when two consecutive fixes agree, or when one fix with confidence ≥ 0.6 arrives while the estimate is within 6 m of a stairs/lift node of the old floor. If no fix arrives for 90 s the position is flagged `approximate`.
`MapMatcher`	Projects the filtered position onto the nearest walkable edge (`walk`, `ramp`) of the same floor within 4 m. When a heading is available, among candidate edges at most 1.5 × the nearest distance away it prefers those within ±45° of the heading. If no edge is within 4 m the position is returned unsnapped. Result and its `nodeId` (nearest node on the edge) go to `PositionManager`. A hidden-Markov matcher (Newson & Krumm 2009) is possible future work.
`NavGraphCache`	Holds the public navigation graph from `GET /buildings/{code}/nav-graph` in memory and IndexedDB; offers `nearestNode` and `edgesOnFloor`. Restricted nodes and edges are not part of the public graph.

3.4.5 Training, evaluation and publishing (server)

Stage	Specification
Collection (Android)	Plug-in `WifiScanner.scan()` via `WifiManager.getScanResults()`. On Android 9+ foreground scans are throttled to four per two minutes; data-collection phones disable “Wi-Fi scan throttling” in developer options. Reference grid every 3 m in corridors plus room centres, doors, stairs, lifts, entrances; at least 10 scans per point, two headings, two times of day, at least two phone models. The point label, floor, `(x, y)` and device are stored with every sample. Only AP identifiers and signal strengths are recorded — no device identifiers, no traffic.
Feature construction	AP universe = BSSIDs observed in ≥ 10 % of training samples. Missing AP = −100 dBm, values below −90 clipped. Per-point training vector = median over its scans; raw scans may be added as extra rows. Device-invariant variants are evaluated (pairwise RSSI differences, rank order); the winner is exported as `featureMode` and must be implemented by the TypeScript estimator. `raw` is mandatory, other modes only if they win by a clear margin.
Model selection	`RadioMapBuilder` runs `GridSearchCV` with `GroupKFold` where the group is the reference point (random splits would leak neighbouring scans and overstate accuracy) over k ∈ {1, 3, 5, 7}, metric ∈ {Euclidean, Manhattan, cosine} and the feature variant. The chosen values are written into the radio map; scikit-learn’s `KNeighborsClassifier` (floor) and `KNeighborsRegressor` (x, y) are used for evaluation and for `ServerKnnPositioner`.
Metrics	`AccuracyEvaluator`: mean/median/p75/p90 position error in metres, error CDF, floor accuracy, per-device error. Acceptance for the Test Plan: median ≤ 3 m, p90 ≤ 5 m, floor accuracy ≥ 95 % on held-out points collected on a different day. If the target is missed the report states the measured values and the hybrid strategy (QR anchors) compensates.
Device heterogeneity	Different phones report RSSI shifted by several dB. Compared in the evaluation: multi-device training data, pairwise-difference features, per-device offset calibration. The best variant becomes the default.
Publishing	`RadioMapRegistry.publish(building, map, force)` accepts a new version only if floor accuracy ≥ old − 2 percentage points and median error ≤ 1.1 × old (or `force`); otherwise the old version stays active (rollback requirement from D6). The new row in `radio_map_versions` and the `RadioMapPublished` event are written in one transaction. Clients see the new `ETag` on their next check.
Staleness	`CalibrationService` compares `access_points` with each new survey: if more than 30 % of recently observed BSSIDs are unknown or known APs have not been seen for 14 days, `/health` reports `radioMaps.staleSuspected` and the admin screen suggests re-calibration (APs moved or replaced).

Server-side classes

```
class RadioMapBuilder:      def build(self, building: str, samples: list[Sample]) -> RadioMap
                            def export(self, m: RadioMap) -> dict            # client JSON format (Section 3.2)
class AccuracyEvaluator:    def evaluate(self, m: RadioMap, test_set: list[Sample]) -> ModelMetrics
class RadioMapRegistry:     def current(self, building: str) -> RadioMapVersion | None          # immutable
```

```
      snapshot
                            async def publish(self, building: str, m: RadioMap, *, force: bool = False) ->
                                  ModelMetrics
                            async def activate(self, building: str, version: str) -> None       # rollback
class ServerKnnPositioner:  async def estimate(self, req: PositionRequest) -> PositionEstimate   # reference;
      flag off in prod
    # postcondition: (x, y) lies inside the extent of the returned floor, or fallback_required is True
```

Parity test. The same 200 held-out scans are fed to `ServerKnnPositioner` and to the TypeScript `LocalKnnEstimator` (run with Vitest); the build fails if floors differ or positions differ by more than 0.1 m. This is what makes the duplicated estimator safe (T14).

Privacy. The scan, the position and the filter state exist only in the memory of the phone. Training data are AP measurements at known coordinates taken by team members and contain no personal data. The radio map and navigation graph are public and contain no user data.

3.5 Routing subsystem (SS-05)

Responsibility. Compute the optimal multi-floor route between two points on the campus graph, optionally restricted to step-free paths, and turn it into human-readable instructions. **Owner:** Taci. **Requirements:** FR-04, FR-05, NFR-01 (route computation ≤ 500 ms, T-74).

3.5.1 Graph model

`NavGraph` wraps an undirected `networkx.Graph`. Node attributes: `floor`, `x`, `y`, `kind` (`corridor|junction|door|stair|elevator|entrance|connector`), `accessible`, `restricted`, `ref_code`. Edge attributes: `weight_m`, `kind` (`walk|ramp|stairs|elevator|outdoor`), `accessible`, `restricted`, `floors_spanned`. Stair and lift shafts are vertical chains of nodes with identical `(x, y)` on consecutive floors, joined by `stairs`/`elevator` edges. Inter-building links (for example the D–C connection on basement level) are `outdoor` or `walk` edges. Expected size: D block ≈ 7 floors × 60 nodes; whole campus < 3 000 nodes, so memory use is negligible. **Restricted or hazardous areas** marked by an admin are excluded in every cost model (health and safety constraint of D2) and are not part of the public graph export.

NavGraph, CostModel, RoutePlanner

```
class NavGraph:
    async def load(self, repo: NavRepository) -> None            # builds G; validates integrity rules
    def invalidate(self) -> None                                  # called on GraphInvalidated
    def nearest_node(self, floor: int, x: float, y: float, max_dist_m: float = 10.0) -> NodeId
        # raises PositionOffGraph if no node within max_dist_m on that floor
    def view(self, cost: CostModel) -> nx.Graph                   # filtered view (accessibility, restricted)
    def export_public(self, building: str) -> PublicGraph        # {version, nodes[], edges[]} for GET
        /nav-graph

class CostModel(Protocol):
    def edge_cost(self, u: NodeAttrs, v: NodeAttrs, e: EdgeAttrs) -> float | None   # None = edge forbidden
class StandardCost:    # weight_m + 8 m per floor on stairs + 10 m per lift ride (waiting); restricted -> None
class AccessibleCost:  # None if edge or either node is not accessible or is restricted; lift ride +10 m

class RoutePlanner:
    def __init__(self, graph: NavGraph, instructions: InstructionGenerator): ...
    def plan(self, start: NodeId | Position | LocationId, goal: LocationId, *, accessible: bool, lang: str =
        'en') -> Route
    # raises RouteNotFound | NoAccessibleRoute | PositionOffGraph
```

3.5.2 Algorithm

1. **Resolve the start.** A `nodeId` is used as is. A `locationId` becomes that location's `door_node_id`. A position (fallback form) inside a room polygon snaps to the room's `door` node, otherwise to the nearest node on the same floor within 10 m. The goal is the `door_node_id` of the target location.
2. **Search.** `networkx.astar_path` on `graph.view(cost)` with edge cost from the active `CostModel`. Heuristic: Euclidean distance in the floor plane, ignoring floor difference (admissible because every edge cost ≥ its planar length). **Routes that cross buildings use `h = 0` (Dijkstra)** because local building coordinates are not in a common frame; correctness is preferred over speed there.
3. **Failure analysis.** If no path exists and `accessible=True`, the planner re-runs with `StandardCost`; if that succeeds it raises `NoAccessibleRoute` (the API returns 409 with the standard route attached under `details.alternative`), otherwise `RouteNotFound`.

4. **Segmentation.** The node path is split into segments whenever the floor, edge kind or heading changes by more than 30°.
5. **Instructions** (`InstructionGenerator.build`). For consecutive segment headings θ_1, θ_2 (y-axis up) the turn angle is $\Delta = \text{wrap}(\theta_2 - \theta_1)$ in (−180°, 180°]. $|\Delta| < 30°$ straight, $30°…150°$ left if $\Delta > 0$ else right, $> 150°$ turn around. Segments shorter than 2 m merge into their neighbour. Stairs and lifts become one step each ("Take the stairs up 2 floors"). Text comes from i18n templates (EN/TR); the client may speak it (FR-09) and vibrate once for left, twice for right (FR-11).
6. **ETA.** `distance / 1.2 m/s` (0.9 m/s in accessible mode) + 6 s per stair flight + 30 s per lift ride.

Complexity. A* is O(E log V) in the worst case; for the expected graph sizes it completes in milliseconds. Targets: p95 < 50 ms for 5 000 nodes, whole `/route` request < 500 ms including snapping and serialisation (to be measured).

Route (value object, JSON-serialisable)

```
class RouteStep(BaseModel):
    index: int; type: Literal['walk','stairs','elevator','turn','arrive']
    text: str; text_tr: str; distance_m: float; floor: int
    turn: Literal['left','right','straight','around'] | None
    polyline: list[tuple[float,float]]; end_node: int                # end_node lets the client advance steps
class Route(BaseModel):
    distance m: float; eta sec: int; floors involved: list[int]; accessible: bool
    steps: list[RouteStep]; geojson: dict           # one LineString feature per floor
    explanation: str | None                           # e.g. 'Avoids stairs: uses the lift in D block, ground
          floor'
```

Tests. Corridor graph; multi-floor stairs; accessible mode avoids stairs; building without lift returns `NoAccessibleRoute`; restricted edge never used; start equals goal; disconnected component; property test (`hypothesis`): A* cost equals Dijkstra cost on random graphs; `graph_check` fails the build on unreachable rooms.

3.6 Campus data and search (SS-07, FR-07)

3.6.1 Domain classes

The D4 hierarchy `Location` → `Room`, `Office`, `Facility` is modelled by one `Location` entity whose `type` (and, for facilities, `subtype`) selects the group: **Room** = `classroom`, `lab`, `amphi`, `studio`, `meeting`; **Office** = `office` (with `occupants`); **Facility** = `facility`, `wc` (with `opening_hours`). One table and one class keep search, routing and administration uniform.

Domain classes (Pydantic)

```
class Building(BaseModel):  code: str; name: str; name_tr: str; heading_offset_deg: float; floors: list[Floor]
class Floor(BaseModel):     id: int; building code: str; level: int; label: str; svg url: str; geojson url: str
                            width_m: float; height_m: float; meters_per_unit: float
class Location(BaseModel):  id: int; code: str; name: str; name tr: str; type: LocationType; subtype: str |
None
                            building code: str; floor: int; x: float; y: float; door node id: int | None
                            is accessible: bool; is active: bool; opening hours: OpeningHours | None;
occupants:
                                  list[PersonRef]
class Person(BaseModel):    id: int; full name: str; title: str | None; role: str; department: str; faculty:
str
                            email: str | None; office hours: OpeningHours | None; location id: int | None


class OpeningHours(BaseModel):                        # JSON in locations.opening_hours / people.office_hours
    weekly: dict[Weekday, list[tuple[time, time]]]     # e.g. {'mon': [('08:30','22:00')], ...}
    exceptions: list[ExceptionDay]                     # holidays: date, closed | alternative intervals
    def is open(self, at: datetime) -> bool            # evaluated in Europe/Istanbul
    def closes_at(self, at: datetime) -> time | None   # None when closed or open-ended
    def next_open(self, at: datetime) -> datetime | None
```

3.6.2 Repositories and search

Repositories and SearchService

```
class CampusRepository:                                  # all methods async, parameterised SQL via
     SQLAlchemy
    async def get location(self, id: int) -> Location | None
    async def get_by_code(self, code: str) -> Location | None
    async def search(self, q folded: str, *, type: str|None, building: str|None, limit: int) ->
          list[Scored[Location]]
    async def get_person(self, id: int) -> Person | None
    async def search people(self, tokens: list[str], limit: int) -> list[Scored[Person]]
    async def find_nearest(self, origin_node: int, subtype: str, accessible: bool, limit: int = 3) ->
          list[Location]
    async def list buildings(self) -> list[Building]; async def list floors(self, building code: str) ->
          list[Floor]
```

```
class NavRepository:        all_nodes(building) / all_edges(building) / replace_graph(building, nodes, edges)
class RadioMapRepository:   get active(building) / add version(...) / list versions(building) /
      activate(building, version)
class FingerprintRepository: add_samples(points, samples) / samples_for(building) / ap_inventory(building)
class AdminRepository:      users, refresh tokens, audit rows

class SearchService:
    async def search(self, q: str, *, type=None, building=None, limit=8) -> list[SearchHit]
```

Search algorithm. (1) `q_folded = fold_tr(q)`; (2) exact code match scores 1.0; (3) trigram similarity `search_text % q_folded` with threshold 0.25 (the `pg_trgm` GIN index keeps this fast); (4) people matched on whole tokens; (5) final score `0.6·similarity + 0.3·type prior + 0.1·proximity` (proximity only when an origin node is supplied; distance measured on the graph); (6) ties broken by code. Occupants are joined from `people.location_id`. The admin interface writes `search_text` through the same `fold_tr` so queries and data always agree.

3.7 Administration and authentication (SS-06)

Authorised staff keep campus data up to date (FR-08). Students and visitors never log in (FR-12). Accounts are created with `tools/create_admin.py` (or the one-time seed variables, Section 2.5); there is no public registration.

Role	May do
`editor` **(for example a department coordinator)**	Create, change and deactivate locations; change opening hours and office assignments; edit people; read the audit log of their own actions.
`admin`	Everything an editor may do, plus: import/replace navigation graphs, import fingerprints, rebuild/publish/roll back radio maps, read the full audit log.

AuthService, CampusAdminService, CalibrationService, AuditLogger

```
class AuthService:
    def hash password(self, plain: str) -> str                  # Argon2id
    async def login(self, username: str, password: str) -> TokenPair      # raises InvalidCredentials
    async def refresh(self, refresh token: str) -> TokenPair              # rotation: old token revoked
    def verify access(self, token: str) -> AdminClaims                    # HS256 pinned; checks exp, iat, sub,
          jti, role

class CampusAdminService:
    async def upsert location(self, admin: AdminClaims, data: LocationIn) -> Location
    async def delete location(self, admin: AdminClaims, id: int) -> None  # 409 if referenced
    async def set_hours(self, admin: AdminClaims, id: int, hours: OpeningHours) -> Location
    async def assign office(self, admin: AdminClaims, person id: int, location id: int) -> Person
    async def import_graph(self, admin: AdminClaims, building: str, data: NavGraphIn) -> ImportReport   # role
          admin

class CalibrationService:                                  # UC-07; role admin
    async def import survey(self, admin: AdminClaims, csv: UploadFile) -> ImportReport
    async def rebuild(self, admin: AdminClaims, building: str) -> ModelMetrics        # builder + evaluator, no
          publish
    async def publish(self, admin: AdminClaims, building: str, version: str, *, force: bool = False) ->
          ModelMetrics

class AuditLogger:  async def record(self, admin_id: int, action: str, table: str, record_id: int, before:
dict,
      after: dict) -> None
```

Rules. Access tokens live 15 minutes; refresh tokens are 256-bit opaque random values stored only as SHA-256 hashes, valid 7 days, single use. Every mutating call writes an `admin_actions` row with a before/after diff in the same transaction as the change. Changes that touch navigation nodes/edges publish `GraphInvalidated`; changes to locations take effect on the next request (FR-08: “without code-level modification or restart”, scenario 4 of D4). Login is rate-limited (5/min/IP) with a progressive delay per username and IP pair instead of a hard account lock (a lock would let anybody lock out the administrators); failures are indistinguishable between unknown user and wrong password. `publish` only succeeds if the new radio map is not less accurate than the active one (Section 3.4.5).

3.8 API layer and orchestration

NavigationOrchestrator (Facade)

```
class NavigationOrchestrator:
    def   init  (self, parser: IntentParser, resolver: EntityResolver, validation: ValidationLayer,
                 composer: ResponseComposer, planner: RoutePlanner, campus: CampusRepository): ...
```

```
    async def handle_query(self, req: QueryRequest) -> QueryResult
    async def route_to(self, req: RouteRequest) -> Route
```

`handle_query` implements the pipeline of Figure 4: parse → resolve and validate → compose the templated answer → (if the intent is `navigate` and an origin was supplied) plan the route → assemble `QueryResult`. Without an origin the response contains no route and the client calls `POST /route` after it has asked `PositionManager` for the current node. The orchestrator owns no state; all collaborators are injected by FastAPI dependencies created in the lifespan handler. The request's `origin` is passed to the resolver and planner and is dropped when the function returns.

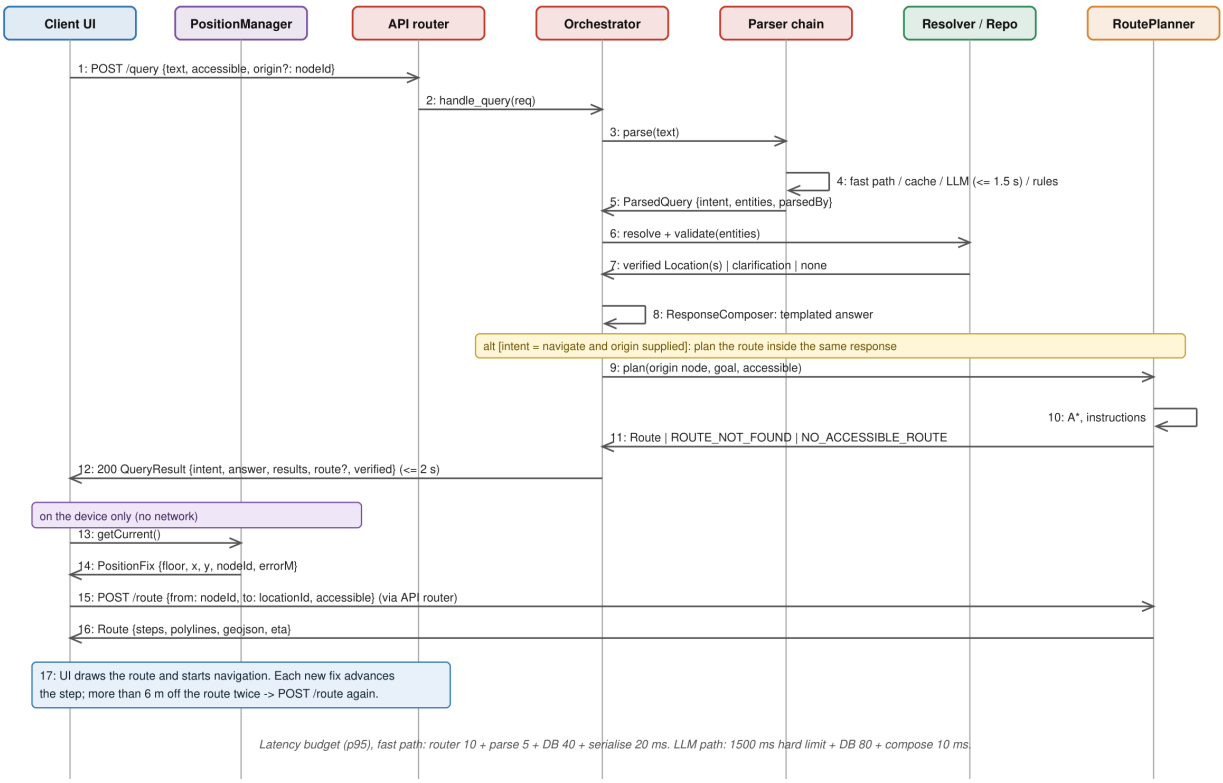
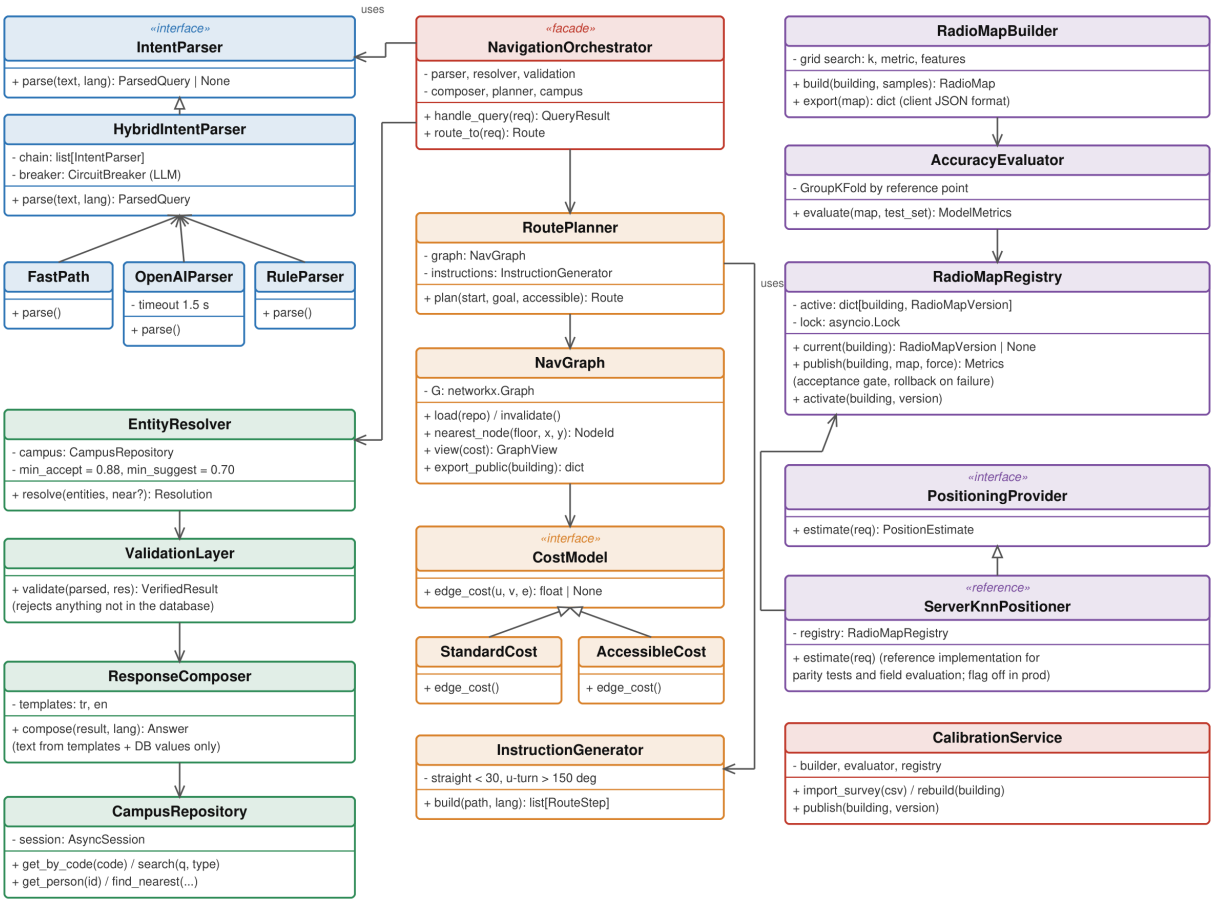


Figure 4 – Sequence diagram of a navigation query

Start-up sequence (lifespan): (1) read and validate settings; (2) create the async engine and run `SELECT 1`; (3) load the navigation graph and run integrity checks; (4) load the metadata of active radio maps (a building without one is allowed and reported by `/health` as `radioMaps: {D: "missing"}`); (5) build the in-memory code and name index for the fast path; (6) create the OpenAI client if a key is configured; (7) mount routers. Failure of step 2 or 3 logs the error and exits with status 1 so the platform restarts the process. **Shutdown:** on SIGTERM finish in-flight requests (30 s), flush the audit log, dispose the engine.

Class overview. Figure 5 summarises the main backend classes and their relations.



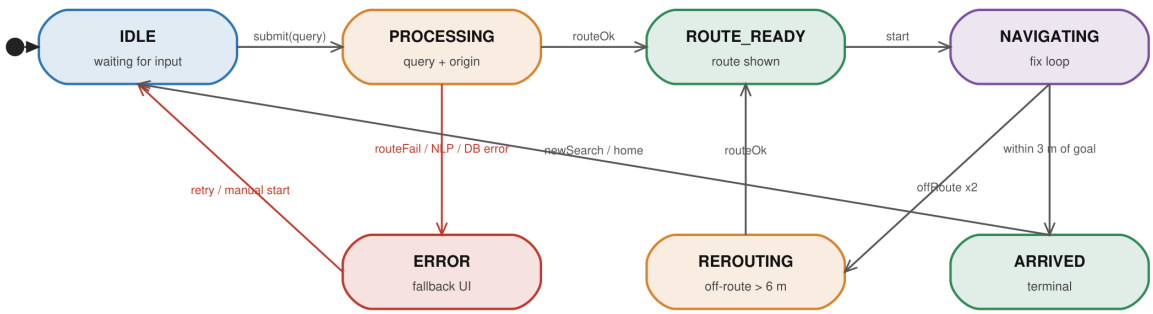
Dashed arrows: realization / dependency. Open arrows: association (has-a / uses). Hollow triangle: implements.

Figure 5 – Backend class diagram

3.9 Client application (SS-01)

Responsibility. Present the user flow Home → Search → Map → Navigation → Arrived (D4 §3.5.5) plus Consent, Settings and Admin screens; render floor plans; run the position pipeline (Section 3.4); speak instructions; keep working through short network outages. **Owner:** Fatima. **Requirements:** FR-06, FR-09, FR-10, FR-11, FR-12, NFR-04.

3.9.1 Session state machine



Guards: submit needs consent.accepted or manual-only mode. Any state -> IDLE on stop. Offline: PROCESSING serves the cache; NAVIGATING continues on the last route.
Events: submit, routeOk, routeFail, start, fix, offRoute, arrive, stop, retry. Transitions are pure reducer functions -> table-driven unit tests (E2E-NAV-xxx).

Figure 6 – Navigation session states

State and actions (TypeScript)

```
type SessionState = 'IDLE'|'PROCESSING'|'ROUTE_READY'|'NAVIGATING'|'REROUTING'|'ARRIVED'|'ERROR';
interface NavigationState { state: SessionState; query: string; results: SearchHit[]; destination:
      Location|null;
  route: Route|null; stepIndex: number; position: PositionFix|null; accessible: boolean; voice: boolean; error:
        AppError|null }
type Action = {t:'submit',q:string} | {t:'routeOk',route:Route,dest:Location} | {t:'routeFail',e:AppError}
            | {t:'start'} | {t:'fix',fix:PositionFix} | {t:'offRoute'} | {t:'arrive'} | {t:'stop'} |
                  {t:'retry'};
function sessionReducer(s: NavigationState, a: Action): NavigationState   // pure; table-driven unit tests
```

Progress rules. `NavigationController` subscribes to `PositionManager`; progress is driven by **fix events**, not by a timer (a WiFi fix arrives about every 30 s, steps in between). While `NAVIGATING`, each new fix is projected onto the route polyline; when the distance to the current step's `end_node` is below 3 m the next step becomes active; when the distance to the destination door node is below 3 m (inside the 3–5 m accuracy threshold of D4) the reducer emits `arrive`. If the distance from the polyline exceeds 6 m for two consecutive fixes the controller dispatches `offRoute`, calls `POST /route` with the current node and returns to `NAVIGATING`; if the server finds no route the session goes to `ERROR` and the user is asked to choose the starting point manually. Fixes flagged `approximate` never trigger `offRoute`. Without any automatic position, or when the user prefers it, the user advances steps manually ("Next step"), which keeps the demo robust.

3.9.2 Modules

Module	Public interface	Notes
`api/client.ts`	Typed functions generated from OpenAPI (`postQuery`, `postRoute`, `searchLocations`, `getRadioMap`, `getNavGraph`, …); `ApiError` with `code`.	5 s timeout for `/query` and `/route`, 8 s for downloads; one automatic retry for idempotent GETs; 401 → refresh once.
`state/NavigationContext.tsx`	`useNavigation(): [state, dispatch]`	Wraps `sessionReducer`; persists nothing sensitive.
`services/positioning`	`PositionManager`, providers, `LocalKnnEstimator`, `MotionTracker`, `PositionFilter`, `MapMatcher`, `NavGraphCache` (Section 3.4)	Exposed to features only through `usePosition()` (subscribes to `PositionManager`).
`map/FloorMap.tsx`	`<FloorMap floor={FloorMeta} route? position? destination? onRoomClick />`	react-leaflet `MapContainer crs={L.CRS.Simple}`; `SVGOverlay` for the plan; bounds `[[0,0],[heightM,widthM]]`.
`map/RouteLayer.tsx`	`<RouteLayer steps floor activeIndex />`	Polyline per floor, direction arrows, dashed animation (disabled for `prefers-reduced-motion`).
`map/PositionLayer.tsx`	`<PositionLayer fix />`	Dot + accuracy circle of radius `errorM` (dashed and labelled "approximate" when flagged).
`services/speech`	`SpeechRecognizerService.listen(lang): Promise<string>`, `SpeechSynthesizerService.speak(text, lang)`, `isSupported()`	Web Speech API; hidden or replaced by text input when unsupported; recognition audio goes to the browser vendor (stated on the consent screen).
`services/haptics`	`turn('left'|'right')`	`navigator.vibrate(100)` / `[100,80,100]`; on iOS (unsupported) a short sound is played instead.
`services/offline`	`OfflineStore`: `saveRoute`, `getLastRoute`, `saveRadioMap`, `saveNavGraph`, `useOnlineStatus()`	IndexedDB (idb-keyval) + Workbox runtime caching.
`features/consent`	`<ConsentGate>`	Stores only the consent flag/version in `localStorage`; declining forces manual-only mode.
`features/author`	Developer-only graph editor	Click-to-place nodes/edges on a plan, exports `data/nav/<block>.json`.
`i18n`	`t(key, vars)`; languages `tr`, `en`	Instruction templates, floor names, room types.

3.9.3 Consent and permissions

The Consent screen (KVKK, NFR-06) appears before the first search and names every processor: query text goes to our API and, only when the rules cannot decide, to OpenAI; voice audio is handled by the browser vendor's speech service; the WiFi scan, motion sensors and QR camera are used **on the device only**; the downloaded radio map and graph are public data. Declining location use leaves manual positioning available. Android permissions are requested only when needed: `ACCESS_FINE_LOCATION` and `NEARBY_WIFI_DEVICES` (Android 13+) for scanning, `CAMERA` for QR; step detection from the accelerometer needs no extra permission. Sensors and scanning run only while a navigation session is active and the screen is on, which limits battery use.

3.9.4 Map viewer

Each floor has an SVG (cropped from the official wayfinding PDF, header and legend removed), a `rooms.json` of anchors (and polygons where available) and a GeoJSON of features. Metres map to Leaflet units one-to-one so that the accuracy circle and route distances are physically meaningful. A floor switcher highlights the floors the route traverses; the active step's floor is selected automatically. The campus overview is a schematic SVG with entrances (Aksu, İncesu) and buildings A–K and the Rectorate; blocks without a plan show "floor plan coming soon" while search still works.

3.9.5 Offline behaviour (FR-10)

Resource	Strategy (Workbox)	Rationale
App shell, fonts, icons	Precache	Loads with no network.
`/buildings`, floor metadata, SVG/GeoJSON plans	StaleWhileRevalidate	Plans change rarely; instant display.
`/nav-graph`, `/radio-map`	StaleWhileRevalidate + copy in IndexedDB (`ETag` revalidation)	Positioning, map matching and node lookup keep working offline.
`/route`, `/query` responses	NetworkFirst, 3 s timeout, then cache; last route also in IndexedDB	Navigation continues for 30–90 s without network.
`/admin/*`	NetworkOnly	Never cache privileged data.

An offline banner states that the last route is shown; positioning continues entirely on the device (WiFi, QR, manual, dead reckoning); re-routing is unavailable until the connection returns, and the banner says so; NLP falls back to a local code lookup against the cached room index.

3.9.6 Screen inventory and accessibility criteria

The interactive prototype (`TEDU Guide.html`, May 2026) fixes the visual language (dark theme, Inter/JetBrains Mono, red accent, cards, badges) and the five-screen flow. The implementation keeps that design but removes everything that was hard-coded: AI result card (now real `parsedBy`/`latencyMs`), fixed route and fixed floor plan, fixed "YOU" marker, inert Accessibility and Voice buttons, fixed user name and recent searches. New screens required by D4/D6 but absent from the prototype: Arrived, Consent, manual position picker, Settings, Admin, offline and error states. **Accessibility acceptance criteria (WCAG 2.1 AA):**

- Contrast ≥ 4.5:1 for text (the prototype's tertiary text colour `#4e5668` on `#080b12` measures about 2.7:1 and will be lightened to ≈ `#7c869b`).
- All controls are native `button`/`a`/`input` elements with accessible names; visible focus ring; complete keyboard operation; touch targets ≥ 44 px.
- Navigation instructions are exposed in an `aria-live="polite"` region; the voice mode reads each new instruction; accessibility mode switches voice on by default (Scenario 3 of D4).
- Respect `prefers-reduced-motion`; the layout works at 200 % zoom.
- Verified with axe-core and Lighthouse (score ≥ 90) and a manual screen-reader pass (TalkBack/VoiceOver).

3.10 Map data pipeline (offline tools)

TED University publishes vector wayfinding maps (designer: Aysima Zümra Kürekci) for blocks A, B, C, D, E, F, H and the Rectorate (A3 landscape, 1683.78 × 1190.55 pt). Each page contains the floor plan, numbered markers and a bilingual legend. Digitising these maps is the largest one-time effort (D2, manufacturability constraint); the pipeline below makes it systematic and repeatable. **Owner:** Fatima (graph review: Taci).

Step	Tool	Output	Verification
1 Crop and clean	`pdf_to_svg.py` (PyMuPDF `get_svg_image`, per-page clip rectangle)	`floorplans/<B>/<level>.svg`	Visual diff against the PDF; viewBox equals plan area.
2 Extract rooms	`extract_rooms.py`: markers from `get_text('dict')` inside the plan area, legend rows pairing number → Turkish/English name	`<level>.rooms.json` (`n, code, name_tr, name_en, type, anchor`)	Marker count equals legend count; codes unique; code regex `[A-H]\d{3}`; codes outside the pattern reviewed by hand.
3 Calibrate	Tape measurement of a corridor/room; shared building origin across floors; compass bearing of one corridor	`meters_per_unit`, origin, `heading_offset_deg`	Stair/lift positions align between floors; heading offset checked by walking the corridor with the app.
4 Author graph	`features/author` editor	`nav/<block>.json` (`nodes[], edges[]`, QR `ref_code`s)	Every room has a door node; every floor has stairs; lifts and ramps verified on site.
5 Validate	`graph_check.py` (CI)	Report	No unreachable room; accessible reachability per floor; edge lengths within 15 % of geometric length; `ref_code` unique.
6 Seed	`build_seed.py`	SQL/JSON seed	Idempotent re-run produces no diff.

Scope (MoSCoW). Must: block D, all floors (−2 … 4). Should: blocks A and B. Could: C, E, F, G, H, K, Rectorate (search works for all; plan view “coming soon” where no plan exists). Maps for G and K were not provided. The existing prototype dataset (213 rooms, 128 with placeholder names such as “Office D101”, and floors above 3 missing) is replaced by data extracted from the official legends.

3.11 Cross-cutting design

3.11.1 Security (NFR-05)

OWASP Top 10 (2021)	Measure
A01 Broken access control	All `/admin/*` routes depend on `require_role`; deny by default; negative tests for every admin route and role.
A02 Cryptographic failures	HTTPS only (HSTS); Argon2id password hashes; JWT secret from environment; refresh tokens stored hashed.
A03 Injection	SQLAlchemy parameterised queries only; Pydantic validation; LLM output constrained by JSON schema and re-validated against the database.
A04 Insecure design	Stateless server, no position storage, rate limits, resolved-entity invariant, on-device positioning.
A05 Security misconfiguration	Strict CORS allow-list, security headers, debug off in production, `/position` disabled by default, secrets never committed (`.env.example` only).
A06 Vulnerable components	`pip-audit`, `npm audit`, dependency pinning, Dependabot.
A07 Authentication failures	Login rate limit with progressive delay, short token lifetime, rotation, generic error messages.
A08 Integrity failures	Locked dependency files, CI-built artifacts; radio map `checksum` verified by the client before use.

OWASP Top 10 (2021)	Measure
A09 Logging failures	Structured logs with request id; audit log for admin actions; **no** origins, query texts or coordinates in logs (log filter, tested).
A10 SSRF	The server makes outbound calls only to the configured OpenAI endpoint.

Additional risk specific to LLMs (OWASP Top 10 for LLM Applications): **prompt injection** (“ignore previous instructions…”). The model has no tools and no data access; its output is a JSON object validated by schema, then every entity is looked up in the database, and the answer text comes from templates. The worst outcome of a successful injection is an `unknown` intent. The public radio map lists AP identifiers that are broadcast over the air anyway; it contains no user data.

3.11.2 Privacy and data flow (NFR-05, NFR-06)

Data	Origin	Leaves the device?	Stored?	Legal basis / note
Query text	User	Yes, to our API; to OpenAI only on the LLM path	No (cache keeps only the parsed result, TTL 1 h)	Consent screen names the processor.
Voice audio	Microphone	Handled by the browser’s speech service	No	Consent screen discloses this; text input always available.
WiFi scan (RSSI vector)	Android	**No**	No	Used only by `LocalKnnEstimator` in memory.
Motion sensors, QR camera frames, manual taps	Device	**No**	No	Processed in memory; camera frames are never stored.
Position fix	Device	**No**	No	Never leaves the phone.
Origin node id (in `/query`, `/route`)	Device	Yes, only when a route or nearest facility is requested	No (not stored, not logged)	As precise as the corridor network; replaces coordinates (T17).
Radio map, navigation graph, plans	Server	Downloaded by the device	On the device (cache)	Public campus data, no user data.
Fingerprint training samples	Team’s own phones	Uploaded by team members only	Yes (training data)	Contains AP identifiers and signal strengths, no personal data; IT approval requested (backlog P0-08).
Admin credentials, audit log	Staff	Yes	Yes (hash only for password)	Needed for accountability.
Consent flag	Client	No	`localStorage` only	Version-stamped; re-asked when the notice changes.

3.11.3 Performance and capacity (NFR-01, NFR-02, NFR-03)

Path	Budget (p95)	Main contributors
`/query`, fast path	< 200 ms	parse 5 ms, DB 40 ms, compose 10 ms, serialisation 20 ms
`/query`, LLM path	< 2 000 ms	LLM ≤ 1 500 ms (hard limit, typically 800–1 200 ms), DB 80 ms, compose 10 ms; on timeout the rules add 50 ms (worst case about 1 700 ms)
`/route`	< 500 ms	snap 10 ms, A* ≤ 50 ms, instructions 20 ms, DB lookup 40 ms
`/locations/search`	< 150 ms	trigram index lookup
`/nav-graph`, `/radio-map`	< 300 ms (or 304)	pre-serialised JSON, `ETag`; downloaded rarely

Path	Budget (p95)	Main contributors
On-device estimate	< 20 ms per scan	about 75 000 operations (Section 3.4.3), measured on a mid-range phone

Concurrency target: 100 simultaneous users (NFR-02) on one async process. Because positioning is on the device, the steady load is only searches and routes, not a position request every few seconds per user; all I/O is `await`-based and the only CPU-bound work (A*) takes milliseconds. Radio-map training runs in a worker thread. The database pool is 10 connections. Verified later with Locust/k6 (Test Plan). Free-tier cold starts (30–60 s) are mitigated by a keep-alive ping and by warming the service before demonstrations; they are a known limit for NFR-03.

3.11.4 Error handling matrix

Failure	Detection	Behaviour
OpenAI timeout / error / quota / breaker open	`wait_for`, SDK exception, breaker state	Rule parser; `parsedBy: rules`; notice “AI temporarily unavailable”.
Query matches nothing	`Resolution.kind == none`	“Not found” plus up to three suggestions; never a guess.
Ambiguous query	`Resolution.kind == ambiguous`	Clarification card with ≤ 3 options.
Fewer than 3 known APs / low confidence	`estimate()` returns `null` or `conf < 0.25`	Filter prediction continues; banner suggests QR or manual.
Radio map missing or download failed	404 `RADIO_MAP_NOT_FOUND`, network error	Previous version used; otherwise WiFi provider disabled, QR/manual remain.
Motion sensors denied or inaccurate	Permission result, sensor accuracy flag	Dead reckoning off; dot stays at the last fix with growing circle; “Next step” button.
Position cannot be snapped	`PositionOffGraph`	Manual pin-drop prompt.
No route / no accessible route	`RouteNotFound` / `NoAccessibleRoute`	Explicit message; alternative offered (accessible case).
Network loss while navigating	`online/offline` events, fetch failure	Continue on cached route and local positioning; banner; resync on reconnect.
Database unavailable	Pool error	`/health` reports `db: down`; 503 with request id; client shows retry.
Radio-map training fails or is worse	Registry gate	Previous version stays active; admin sees the metrics comparison.

3.12 Build, deployment and operations

Quality gates run on every pull request (GitHub Actions); `main` is protected, needs one approval from another area owner and a green pipeline. Deployment is automatic from `main` after the gates pass.

Job	Steps	Fails the build when
backend	`ruff`, `mypy --strict services/`, `import-linter`, `pytest` (unit + integration against a PostGIS service container), `pip-audit`, export of `docs/openapi.json`	a rule is violated, coverage < 80 %, or the exported OpenAPI differs from the committed one without a contract PR
frontend	`eslint`, `tsc --noEmit`, `openapi-typescript` regeneration, `vitest` (including the radio-map **parity test**), `vite build`, Playwright E2E with a mock API, axe check	types do not match the contract, a test fails, coverage < 70 %, or an accessibility violation of level “serious” appears
data	`graph_check.py` on every `data/nav/*.json`; `build_seed.py` idempotency check	a room is unreachable, a `ref_code` is duplicated, or the seed re-run changes rows
deploy	Vercel builds `frontend/dist`; Render runs `alembic upgrade head` as pre-deploy command and starts the container; smoke test calls `/health`	migration or health check fails (previous release keeps running)

Operations. Logs are structured JSON with a request id and contain no query text, origin or coordinates. `/health` reports `status`, `version`, `db` and the state of each radio map (`ok`, `missing`, `staleSuspected`). An external uptime monitor pings `/health` every five minutes (also keeping the free tier awake); a weekly scheduled job runs `select 1` against Supabase so the project is not paused. Before demonstrations the service is warmed up ten minutes in advance and a local Docker Compose stack is kept as a fallback (risk R-E).

3.13 Requirements traceability

Requirement	Realised by	Verified by (test-id prefix)
FR-01 Indoor positioning	`WifiScanProvider`, `LocalKnnEstimator`, `RadioMapBuilder`, `RadioMapRegistry`, `WifiScanner` plug-in, `PositionManager`	UT-POS, FLD-POS, PAR-POS
FR-02 Accuracy 3–5 m	Grid collection, floor-then-position k-NN, QR anchors, filter and map matching	FLD-POS (error CDF)
FR-03 NLP query	`HybridIntentParser`, `EntityResolver`, `ValidationLayer`, `ResponseComposer`	UT-NLP, EVAL-NLP
FR-04 Route generation	`NavGraph`, `RoutePlanner`, `InstructionGenerator`, `RouteLayer`	UT-RT, IT-API, E2E-NAV
FR-05 Accessibility routing	`AccessibleCost`, `NoAccessibleRoute`, accessibility toggle	UT-RT, E2E-ACC
FR-06 Interactive floor plan	`FloorMap`, map data pipeline	E2E-MAP, UAT
FR-07 Campus DB query	`CampusRepository`, `SearchService`, `people` table, `OpeningHours`	UT-DB, IT-API
FR-08 Admin interface	`AuthService` (roles), `CampusAdminService`, `CalibrationService`, admin screens, audit log	IT-ADM, SEC
FR-09 Voice-only mode	`SpeechRecognizerService`, `SpeechSynthesizerService`, live region	E2E-VOI, UAT
FR-10 Offline	Service worker, Workbox strategies, IndexedDB, on-device positioning	E2E-OFF
FR-11 Haptic (optional)	`haptics.turn`	UAT (Android)
FR-12 Guest access	No authentication on public routes	IT-API
NFR-01 Response time	Fast path, caching, 1.5 s LLM limit, circuit breaker, async I/O, budgets in 3.11.3	PT-LAT
NFR-02 100 users	Stateless async server, pool, on-device positioning	PT-LOAD
NFR-03 99 % uptime	Health checks, keep-alive, restart policy	Monitoring report
NFR-04 Usability	PWA, consent flow, three-tap main path, WCAG	UAT, A11Y
NFR-05 Security	Section 3.11.1	SEC
NFR-06 Privacy / KVKK	Section 3.11.2, on-device positioning, consent gate, log filter	PRIV audit
NFR-07 Maintainability	Package rules, interfaces, CI gates	Static analysis, review
NFR-08 Portability	Docker, standard cloud services, no university-internal dependency	Deployment check
PR-01 … PR-04	Stack as specified (React/Leaflet, FastAPI, PostgreSQL/PostGIS, OpenAI, k-NN, Vercel + Render/Railway)	Review

3.14 Design for testability

The Test Plan (due 20 November) will assign a test id to every case; the design provides the hooks so that each level is possible without special tooling.

Level	What it covers in this design	Tools	Id prefix
Unit	`fold_tr`, parsers with a fake OpenAI client, resolver thresholds, cost models, A*, instruction generator, `sessionReducer`, `LocalKnnEstimator`, `PositionFilter`, `MapMatcher`, `OpeningHours`	pytest, hypothesis, Vitest	UT-NLP, UT-RT, UT-POS, UT-DB, UT-UI
Parity	Same 200 scans through Python and TypeScript estimators	pytest + Vitest on shared JSON fixtures	PAR-POS
Integration	API + PostGIS (testcontainers / compose), admin flows, graph reload, radio-map publish and rollback	pytest, httpx	IT-API, IT-ADM
End-to-end	D4 scenarios 1–4 (first-year student, visitor, wheelchair user, admin edit) with the simulated provider; offline; voice	Playwright	E2E-NAV, E2E-ACC, E2E-OFF, E2E-VOI, E2E-MAP
Evaluation	150+ labelled NLP queries; hallucination property test	`data/eval/nlp_queries.csv`	EVAL-NLP
Field	Positioning accuracy on ≥ 30 held-out points on a different day; per-device error	Android APK, tape measure, CDF script	FLD-POS
Performance / security / accessibility / usability	100 simulated users; OWASP checks; axe and Lighthouse; SUS questionnaire with ≥ 8 people	Locust or k6, OWASP ZAP baseline, axe	PT-LAT, PT-LOAD, SEC, A11Y, UAT

- **Fakes for every external dependency:** in-memory repositories, a fake OpenAI client (timeouts, errors, garbage JSON), a fake `WifiScanner` that replays recorded scans, a simulated walk (`tools/simulate_walk.py`), a fake clock for `OpeningHours`. Tests that call the real OpenAI are marked `@pytest.mark.llm` and are not run in CI.
- **Deterministic data:** `data/seeds` and `data/fingerprints/test` are versioned; synthetic fingerprints (`RSSI = −40 − 10·n·log10(d) + noise`) are allowed for pipeline tests but **never** for reported accuracy.
- **Observable behaviour:** every response carries `parsedBy`, `latencyMs` and `X-Request-ID`; the client logs the active provider and radio-map version in the settings screen for field testing.

3.15 Impact and contemporary issues (preview of the Final Report)

- **Economic:** no new hardware (existing WiFi, printed QR stickers), free hosting tiers, a capped OpenAI budget (development budget USD 10–20; most queries never reach the LLM); the per-query cost is measured and reported in the Final Report.
- **Environmental:** negligible; fewer printed signs and fewer wasted walks; the rules-first chain also avoids unnecessary LLM calls and their energy use. Scanning and sensors run only while navigating to limit battery drain.
- **Societal and inclusion:** step-free routing that states honestly when no accessible route exists, voice and haptic guidance, Turkish and English, no login for visitors, usable by first-year students without training.
- **Ethical and legal:** location is personal data (KVKK 6698); the design keeps it on the device, asks for explicit consent, uses a node id instead of coordinates and logs nothing about position (ACM Code 1.6).
- **Contemporary issues:** reliability and grounding of large language models (hallucination, prompt injection); privacy-preserving location technologies; the platform limits of mobile browsers; reliance on free-tier cloud services.

3.16 Implementation responsibilities and risks

Component ownership is given in Section 2.3. The technical risks tracked in the backlog, with the design measures taken above, are:

Risk	Impact	Mitigation in the design	Owner
R-A WiFi data-collection approval from IT is delayed	Accuracy cannot be measured	Passive measurement with an approval request on file; simulated data for development only; QR and manual providers (T1)	Elif
R-B Accuracy worse than 5 m	FR-02 not met	Honest reporting; floor vote before position; QR anchors every stair and crossing; denser reference grid; filter and map matching	Elif
R-C No WiFi positioning on iOS	Platform coverage	QR and manual on every device; Android APK for the live demo	Fatima
R-D Floor-plan extraction is harder than expected	Delay of every map feature	MoSCoW scope (block D first), semi-automatic extraction, graph editor, validation tool	Fatima
R-E Free-tier cold starts, database pausing	Demo failure	Keep-alive ping, weekly query, warm-up, local Docker fallback, pre-recorded video	Taci
R-F OpenAI quota or cost	NLP degraded	Fast path, cache, 1.5 s limit, circuit breaker, rules fallback, mini model	Elif
R-G Dead reckoning is unreliable (drift, phone in pocket)	Dot jumps or lags	SHOULD priority; QR/WiFi corrections; map matching; fallback to last fix and “Next step” (T15)	Elif / Fatima
R-H Device heterogeneity of RSSI	Different error on different phones	Multi-device training data, pairwise features if they win, per-device error reported	Elif
R-I Battery drain from scanning and sensors	Users stop navigating	Run only while a session is active and the screen is on; 30 s scan interval	Fatima
R-J Duplicate k-NN implementation diverges	Server tests do not reflect the phone	Shared JSON fixtures and a parity test in CI (T14)	Elif / Taci
R-K One-person bottleneck, final exams	Schedule slip	API contract and mocks, weekly integration, critical work finished by 15 December	All

4. Glossary

Term	Definition
A*	Informed graph search using a heuristic; returns an optimal path when the heuristic never overestimates.
Access point (AP) / BSSID	WiFi radio transmitter / its MAC address, used to identify the signal source in a fingerprint.
Anchor	Reference point of a room on the floor plan (marker position from the official map).
Capacitor	Open-source runtime that wraps a web application as a native Android/iOS app and lets it call native plug-ins.
Circuit breaker	Wrapper that stops calling a failing service for a pause (here: the LLM for 120 s after three failures) and uses the fallback instead.
CRS.Simple	Leaflet coordinate system for non-geographic, Cartesian maps.
Dead reckoning (PDR)	Estimating movement from detected steps and heading between absolute position fixes; drifts, so each fix corrects it.
Door node	Navigation-graph node where a room connects to a corridor.
Fast path	Query resolution without LLM when the text is a known room code or exact place name.
Fingerprint	Vector of RSSI values from all visible APs at a known location.
Hallucination	LLM output that is fluent but not grounded in real data (for example a non-existent room).
KVKK	Kişisel Verilerin Korunması Kanunu, Turkish Law No. 6698 on personal-data protection.
k-NN	k-nearest neighbours: predicts from the k most similar training examples.
Map matching	Moving an estimated position onto the nearest walkable corridor of the navigation graph.
Navigation graph	Weighted graph of nodes (corridors, doors, stairs, lifts) and edges (walkable links) used for routing; its public version is also used on the phone.
Parity test	Test that runs the Python and TypeScript k-NN implementations on the same data and requires equal results.
PWA	Web application with a service worker and manifest, installable and partly offline-capable.
QR anchor	A QR code on a wall at a known navigation node; scanning it gives the user’s exact location on any phone with a camera.
Radio map	A versioned database of WiFi signal measurements at known reference points in one building, published as JSON for the phone.
Reference point	A point with known coordinates where WiFi signals were measured during the site survey.
Repository	A class that hides database access from the rest of the system.
RSSI	Received signal strength indicator in dBm; closer to 0 is stronger (−30 very strong, −90 weak).
SRID 0	PostGIS identifier for an undefined (Cartesian) coordinate system.
Stateless server	A server that keeps no per-user data between requests.
Templated answer	Answer text built from fixed templates and database values, never from free LLM text.
Validation layer	Component that checks every entity against the campus database before it reaches the user.
Wayfinding map	Official TED University floor-plan map with numbered legends (PDF).

5. References

[1] ACM. (2018). ACM Code of Ethics and Professional Conduct. https://www.acm.org/code-of-ethics

[2] Aggarwal, P., & Chugh, N. (2022). Review of machine learning techniques for indoor positioning. Journal of Location Based Services, 16(4), 280–322. https://doi.org/10.1080/17489725.2022.2161198

[3] Android Developers. WiFi scanning overview (scan throttling). https://developer.android.com/develop/connectivity/wifi/wifi-scan

[4] Android Developers. Sensors overview. https://developer.android.com/develop/sensors-and-location/sensors/sensors_overview

[5] Bahl, P., & Padmanabhan, V. N. (2000). RADAR: An in-building RF-based user location and tracking system. Proc. IEEE INFOCOM 2000, 2, 775–784. https://doi.org/10.1109/INFCOM.2000.832252

[6] Bruegge, B., & Dutoit, A. H. (2004). Object-Oriented Software Engineering: Using UML, Patterns, and Java (2nd ed.). Prentice Hall.

[7] Capacitor. Documentation and plugin development guide. https://capacitorjs.com/docs

[8] Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9110: HTTP Semantics. IETF.

[9] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[10] Hagberg, A., Schult, D., & Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. Proc. SciPy 2008.

[11] Harle, R. (2013). A survey of indoor inertial positioning systems for pedestrians. IEEE Communications Surveys & Tutorials, 15(3), 1281–1293.

[12] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. Systems Science and Cybernetics, 4(2), 100–107.

[13] IEEE. (2009). IEEE Std 1016-2009, Systems design — Software design descriptions.

[14] IEEE Computer Society. (1999). IEEE Software Engineering Code of Ethics and Professional Practice (v5.2).

[15] ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering.

[16] ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.

[17] Jones, M., Bradley, J., & Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). IETF. And Sheffer, Y., Hardt, D., & Jones, M. (2020). RFC 8725: JWT Best Current Practices.

[18] Butler, H., et al. (2016). RFC 7946: The GeoJSON Format. IETF.

[19] Liu, H., Darabi, H., Banerjee, P., & Liu, J. (2007). Survey of wireless indoor positioning techniques and systems. IEEE Trans. SMC Part C, 37(6), 1067–1080. https://doi.org/10.1109/TSMCC.2007.905750

[20] Newson, P., & Krumm, J. (2009). Hidden Markov map matching through noise and sparseness. Proc. 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, 336–343.

[21] OMG. (2017). Unified Modeling Language (UML) Specification, Version 2.5.1.

[22] OpenAI. (2024). GPT-4 Technical Report. https://openai.com/research/gpt-4 ; OpenAI API documentation (Structured Outputs). https://platform.openai.com/docs

[23] OpenAPI Initiative. (2021). OpenAPI Specification 3.1.0.

[24] OWASP Foundation. (2021). OWASP Top 10; (2019) Application Security Verification Standard 4.0; OWASP Top 10 for LLM Applications.

[25] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825–2830.

[26] PostGIS Project. PostGIS 3 Reference; PostgreSQL Global Development Group. PostgreSQL 15 Documentation (pg_trgm). https://www.postgresql.org/docs/15/

[27] Republic of Turkey. (2016). Kişisel Verilerin Korunması Kanunu — Law No. 6698. Official Gazette No. 29677.

[28] Ramírez, S. FastAPI documentation; SQLAlchemy 2.0 documentation; Pydantic v2 documentation. https://fastapi.tiangolo.com

[29] TED University. Wayfinding and sustainability maps of blocks A–H and the Rectorate (designer: A. Z. Kürekci); TED University campus map.

[30] W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1; W3C Web Speech API Community Group Report.

[31] Workbox (Google). Service worker libraries and caching strategies. https://developer.chrome.com/docs/workbox

[32] Agafonkin, V. Leaflet.js documentation. https://leafletjs.com/reference.html ; React Team. React documentation. https://react.dev